Integraçaõ com IIoT

Professor: Roberto

Aula 01

Cronograma Teórico

01 Introdução ao IIoT

02 Conceito de Microcontrolador

03 Hardware ESP32

04 Começando a Programar IoT

01

Introdução ao IoT e IIoT

Introdução a IIoT

A **Internet das Coisas (IoT)** conecta dispositivos físicos à internet, permitindo a troca de dados entre objetos e sistemas.

IIoT (Industrial Internet of Things) é a aplicação da IoT no contexto industrial, com foco em conectar máquinas, sensores e sistemas em ambientes de produção e infraestrutura. Seu objetivo é aumentar a eficiência, a produtividade e a automação, utilizando dados em tempo real para otimizar processos industriais.

Caracteísticas IoT:

- Comunicação entre dispositivos e sistemas em tempo real.
- Sensores e atuadores para coleta de dados e execução de tarefas.
- Análise de dados para tomada de decisões automatizadas.

Caracteísticas IIoT:

- Integra sensores, máquinas e equipamentos para coletar e compartilhar dados.
- Melhora a produtividade por meio de processos automatizados e análise de dados.
- Utiliza Big Data, Machine Learning e Inteligência Artificial para tomada de decisão.
- Projetado para suportar operações complexas e de larga escala.

Benefícios:

- **Eficiência:** Automação de processos em casa, na indústria e no dia a dia.
- **Sustentabilidade:** Monitoramento de recursos (água, energia).
- **Conveniência:** Controle remoto e integração de dispositivos.

Benefícios:

- **Eficiência:** Automação de processos em casa, na indústria e no dia a dia.
- **Sustentabilidade:** Monitoramento de recursos (água, energia).
- **Conveniência:** Controle remoto e integração de dispositivos.

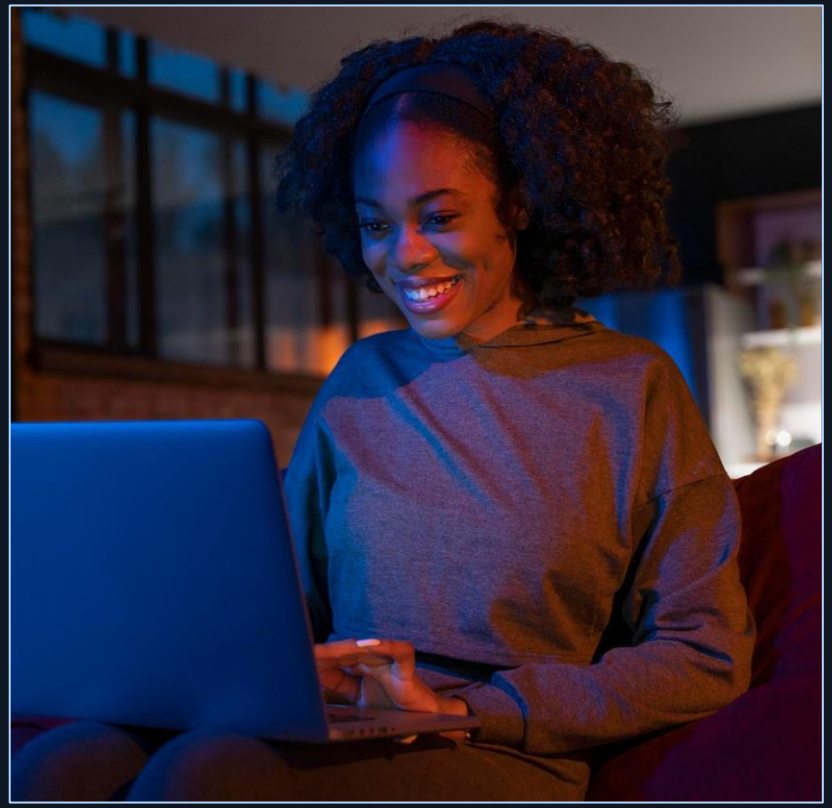
Exemplos:

- Monitoramento de máquinas para prever falhas (manutenção preditiva).
- Gestão de redes de energia (Smart Grid) e monitoramento remoto de equipamentos.
- Rastreamento em tempo real de cargas e frotas.
- Sensores monitorando solo, clima e irrigação para otimizar o cultivo.

Profissionais de TI e a IoT

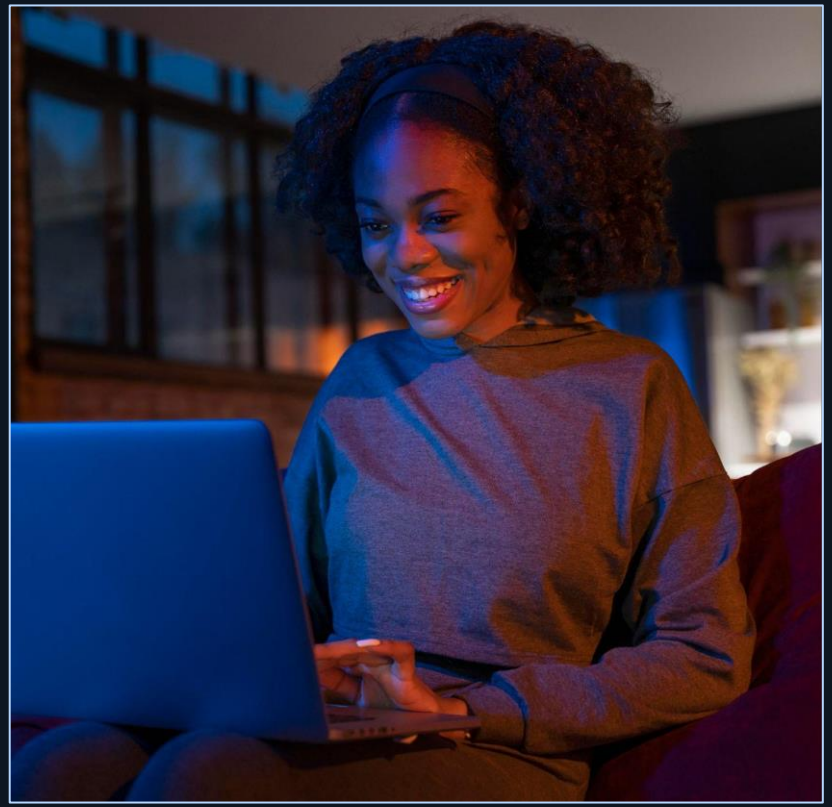
Os profissionais de ADS desempenham um papel essencial na IoT, desenvolvendo os sistemas que conectam dispositivos, processam dados e criam soluções inovadoras.

Sem ADS, a IoT não seria funcional nem escalável.



Contribuições de ADS na IoT

- Desenvolvimento de APIs e sistemas para integração de dispositivos IoT.
- Criação de dashboards para visualização e monitoramento de dados em tempo real.
- Implementação de segurança nos sistemas IoT para proteção de dados.
- Otimização de sistemas para comunicação eficiente e armazenamento de dados.
- Uso de análise de dados para tomada de decisões inteligentes.



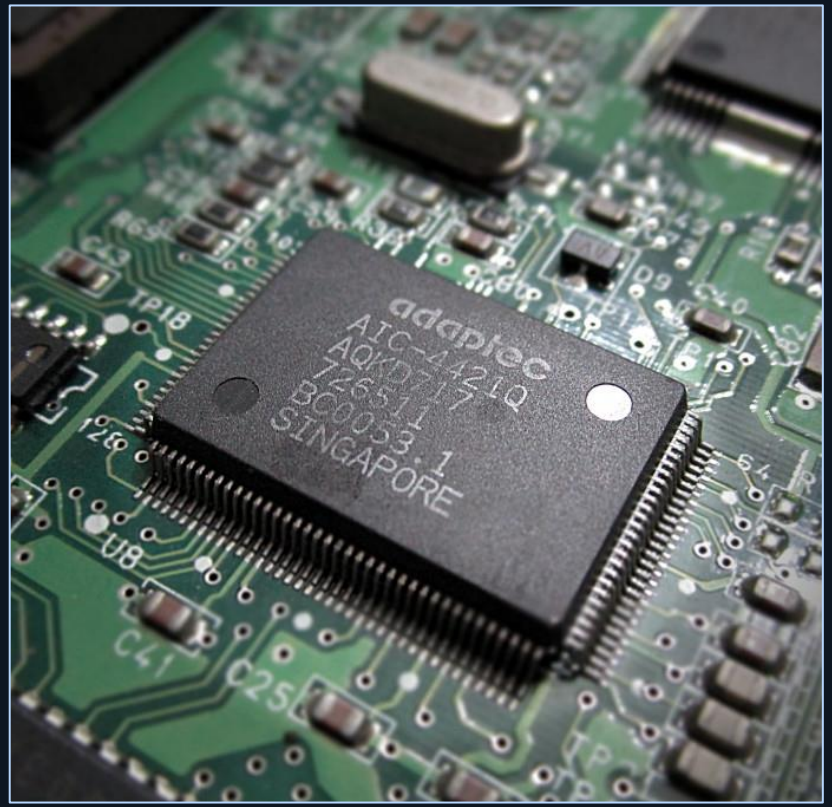
02

Conceito de microcontrolador

O que é o Microcontrolador?

Um microcontrolador é um pequeno computador integrado em um único chip, projetado para executar tarefas específicas, como controlar dispositivos eletrônicos.

- **Contém CPU, memória e interfaces de entrada/saída (I/O).**
- **Amplamente utilizado em dispositivos IoT por sua eficiência e versatilidade.**



Funcionamento

Recebe dados de entradas (sensores, botões).

Processa os dados de acordo com o programa carregado.

Envia comandos para as saídas (atuadores, LEDs, displays).

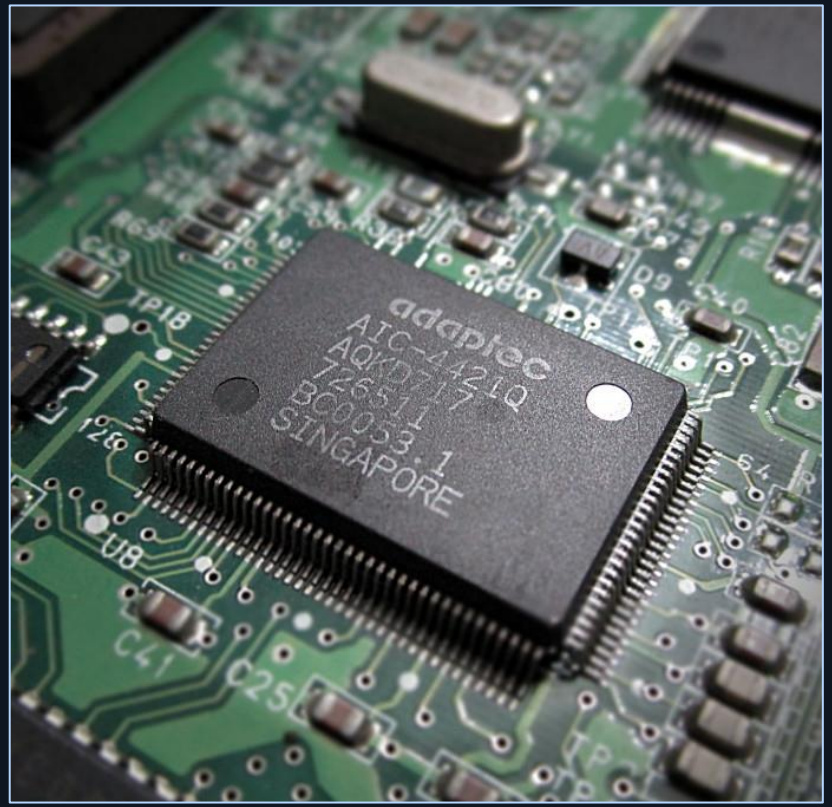
Diferenças entre Microcontrolador e Computador

Tabela comparativa

Característica	Microcontrolador	Computador
Finalidade	Tarefas específicas	Uso Geral
Componentes	Integrados em um único chip	Vários componentes separados
Consumo de energia	Baixo	Alto
Exemplo de Uso	Controle de LEDs, automação IoT	Edição de vídeos, navegação WEB, etc...

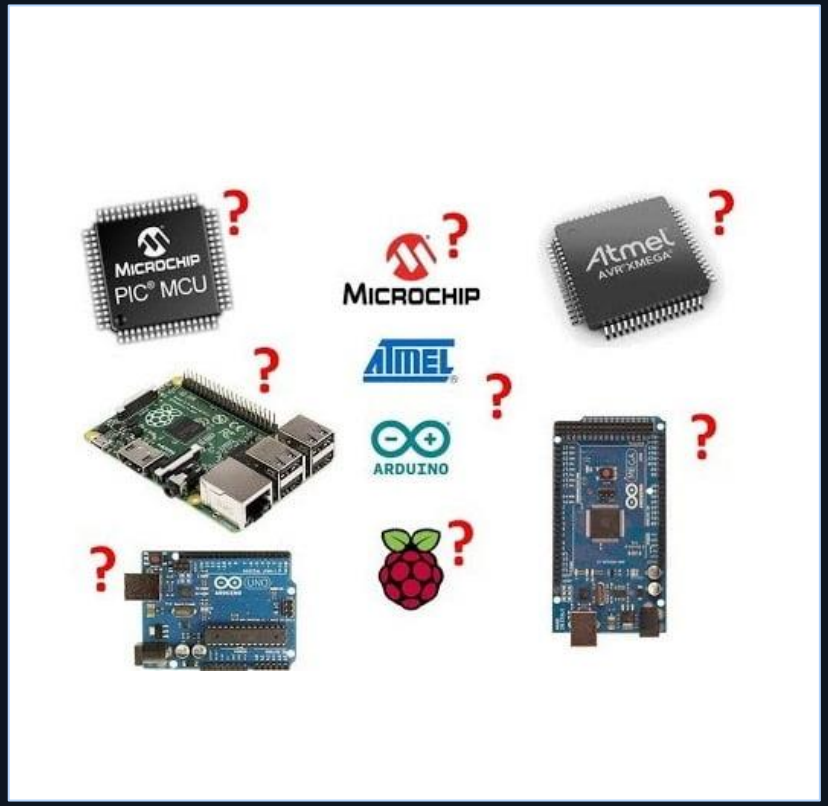
Microcontroladores na IIoT

- **Compactos e Econômicos:** Perfeitos para dispositivos pequenos e baratos.
- **Baixo Consumo de Energia:** Ideais para dispositivos alimentados por bateria.
- **Interface Direta com Sensores e Atuadores:** Facilitam o desenvolvimento de sistemas IoT.



Microcontroladores “famosos”

- **Arduino (Atmega 328);**
- **ESP8266;**
- **ESP32;**
- **Raspberry;**



03

Hardware

Arduino

Arduino é uma plataforma de hardware e software open-source projetada para criar protótipos eletrônicos de forma acessível e prática.

Características Principais:

- **Processador:** ATmega328 (Arduino Uno).
- **Entrada/Saída:** 14 pinos digitais (6 PWM), 6 analógicos.
- **Comunicação:** UART, SPI, I2C.
- **Tensão de Operação:** 5V.

Aplicações: Projetos de automação básica, controle de LEDs, leitura de sensores.

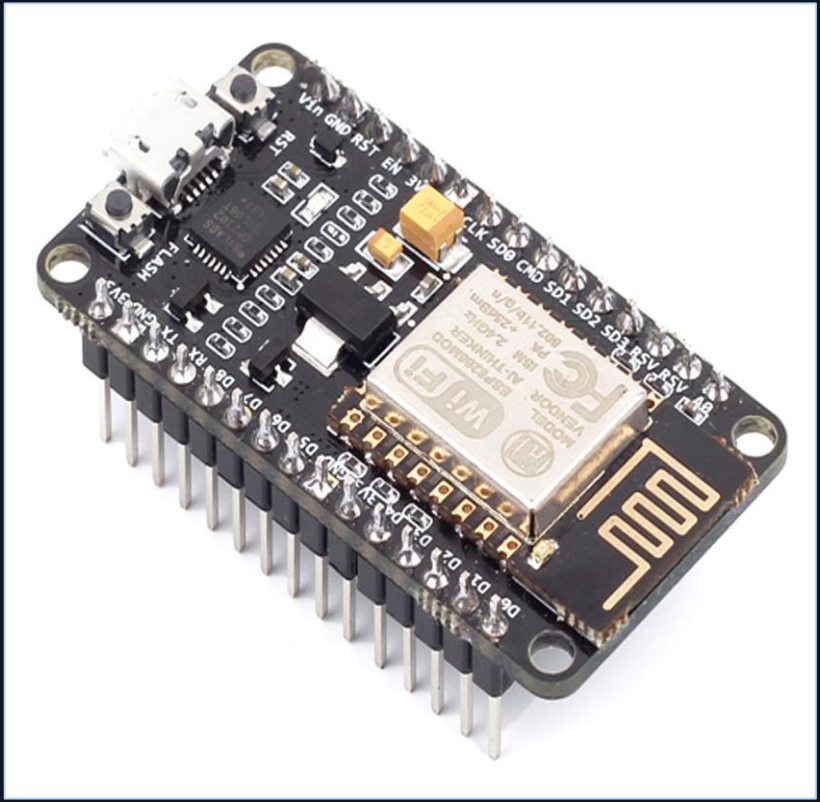
ESP8266

ESP8266 é um microcontrolador compacto com conectividade Wi-Fi, amplamente utilizado em projetos IoT.

Características Principais:

- **Processador:** Tensilica L106, 80 MHz.
- **Conexão:** Wi-Fi (802.11 b/g/n).GPIO: 17 pinos.
- **Memória Flash:** 4 MB (NodeMCU).

Aplicações: Dispositivos IoT conectados, monitoramento remoto.



ESP32

ESP32 é um microcontrolador de alta performance, ideal para aplicações IoT avançadas.

Características Principais:

- **Processador:** Dual-core Xtensa LX6, até 240 MHz.
- **Conectividade:** Wi-Fi, Bluetooth, BLE.
- **GPIO:** Mais de 30 pinos.
- **Tensão de Operação:** 3,3V.

Aplicações: Projetos com múltiplos sensores e conectividade avançada.

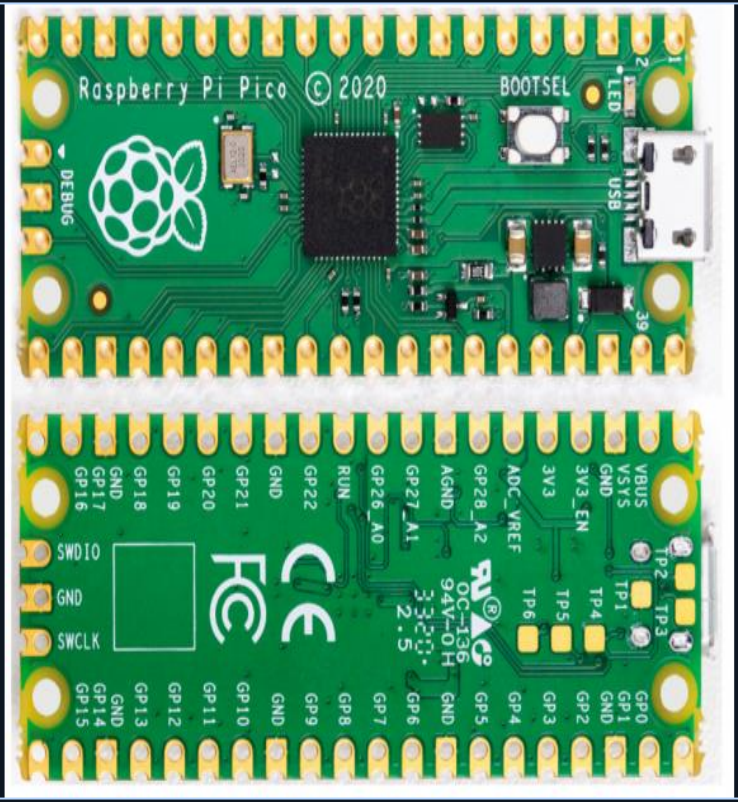
Raspberry pi pico

O Raspberry Pi Pico é um microcontrolador de baixo custo projetado para tarefas simples e eficientes.

Características Principais:

- **Processador:** RP2040 (dual-core ARM Cortex-M0+).
- **GPIO:** 26 pinos.
- **Memória Flash:** 2 MB.
- **Tensão de Operação:** 3,3V.

Aplicações: Controle básico de dispositivos, protótipos IoT compactos.



Raspberry pi 3B

O Raspberry Pi 3B+ é um computador completo em uma placa, ideal para projetos IoT que requerem processamento robusto.

Características Principais:

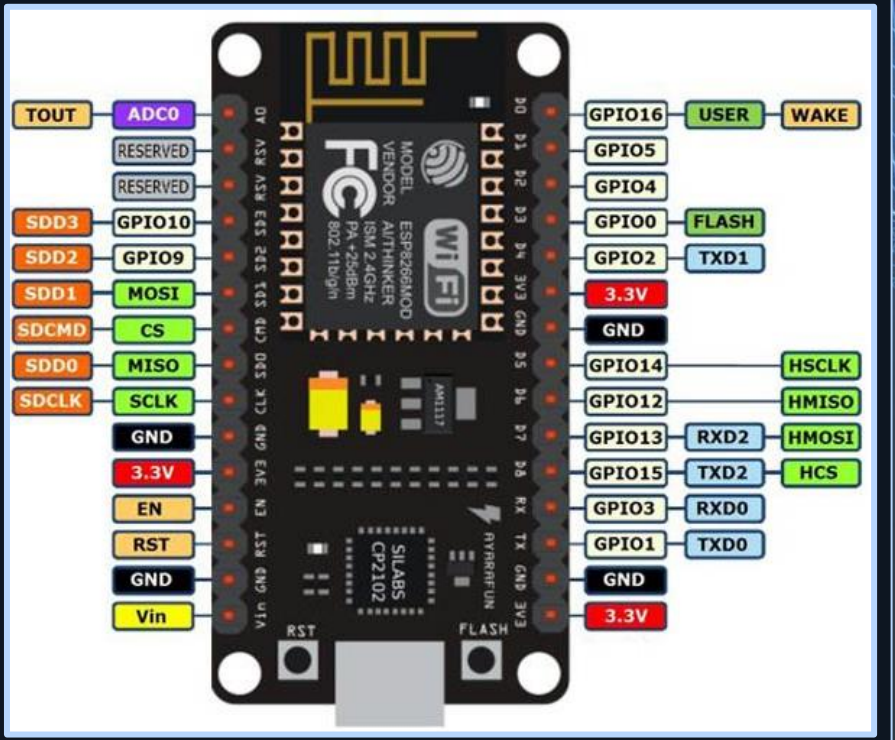
- **Processador:** Quad-core Cortex-A53, 1.4 GHz.
- **Conectividade:** Wi-Fi, Bluetooth, Ethernet.
- **GPIO:** 40 pinos.
- **Sistema Operacional:** Raspbian (Linux).

Aplicações: Servidores web, brokers MQTT, automação residencial avançada.

ESP8266 – Especificações Técnicas

Componentes Principais:

- **Memória Flash:** Armazena o firmware.
- **GPIOs:** Pinos para entrada e saída de sinais.
- **Wi-Fi integrado:** Para conectividade sem fio.
- **Regulador de Tensão:** Aceita alimentação de 5V, converte para 3.3V.
- **Número de Pinos:**11 GPIOs, incluindo entradas analógicas e digitais.



ESP32 – Especificações Técnicas

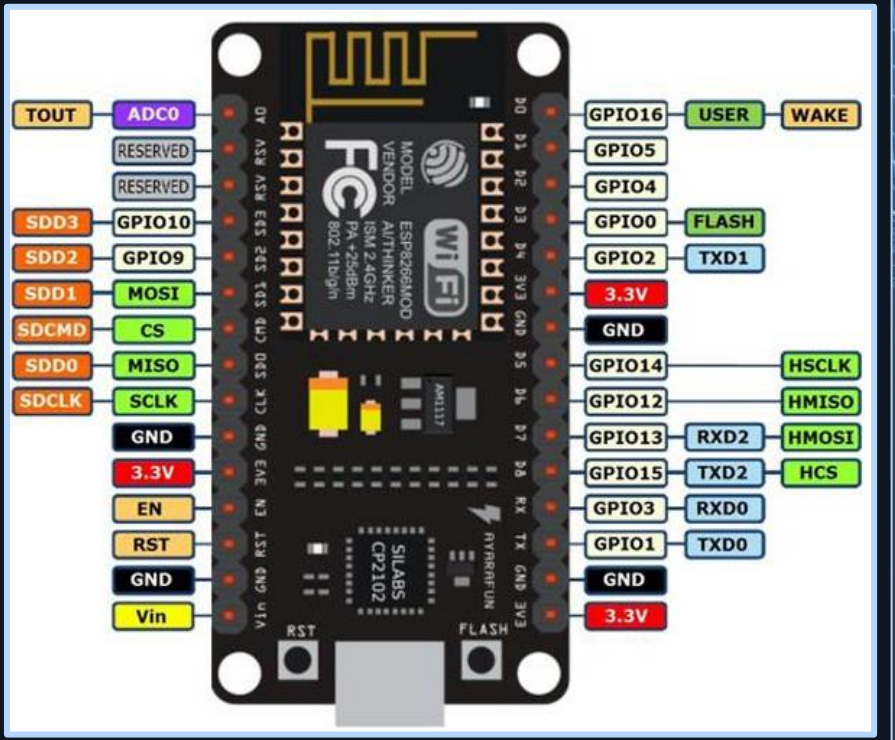
Processador: Tensilica L106, 80 MHz com arquitetura RISC 32bits;

Memória SRAM: 160 KB sendo apenas 64 para código;
Memória Flash: 4MB;

Wi-Fi: 802.11 b/g/n, 2.4 GHz

GPIO: 11 pinos Digitais, 1 canal analógico (ADC0) com 10bits (0 – 1023) resolução

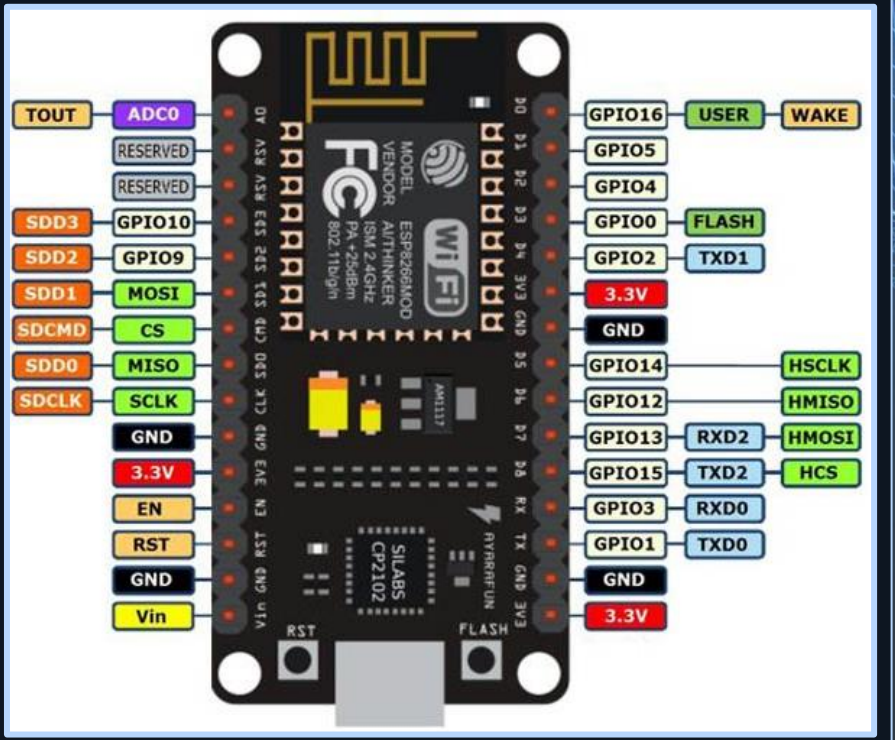
Tensão de Operação: 3,3V



ESP8266 – Funcionamento Básico

Programação:
Arduino IDE, PlatformIO, ou ESP-IDF (SDK oficial).
Linguagens Suportadas: C/C++, MicroPython.

Fluxo de Operação:
Entrada (leitura de sensores ou sinais) → Processamento (código) → Saída (controle de dispositivos).



Ao final dessas aulas gostaria que todos fossem capazes de pensar, planeja e construir um sistema IoT e IIoT

04

Começando a programar o IoT

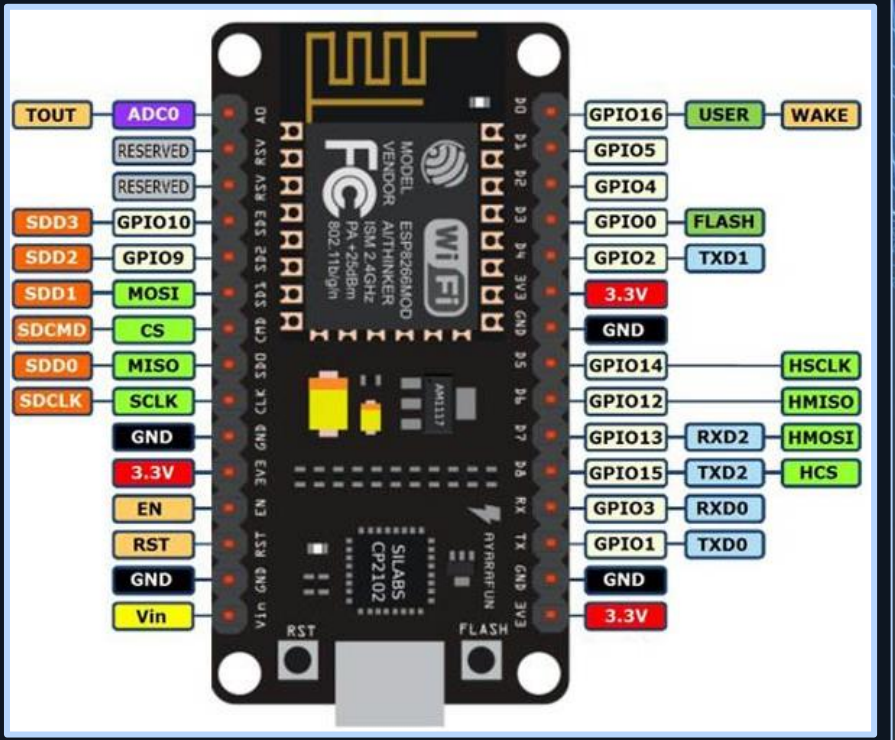
Saídas Digitais

Programação.

IDE:
Nós utilizaremos a IDE do Arduino para fazer a programação do microcontrolador.

Linguagem:
Vamos programar em C/C++

Para a programação temos que utilizar os números da GPIO, para isso vamos entender a sintaxe utilizada pelo Arduino.

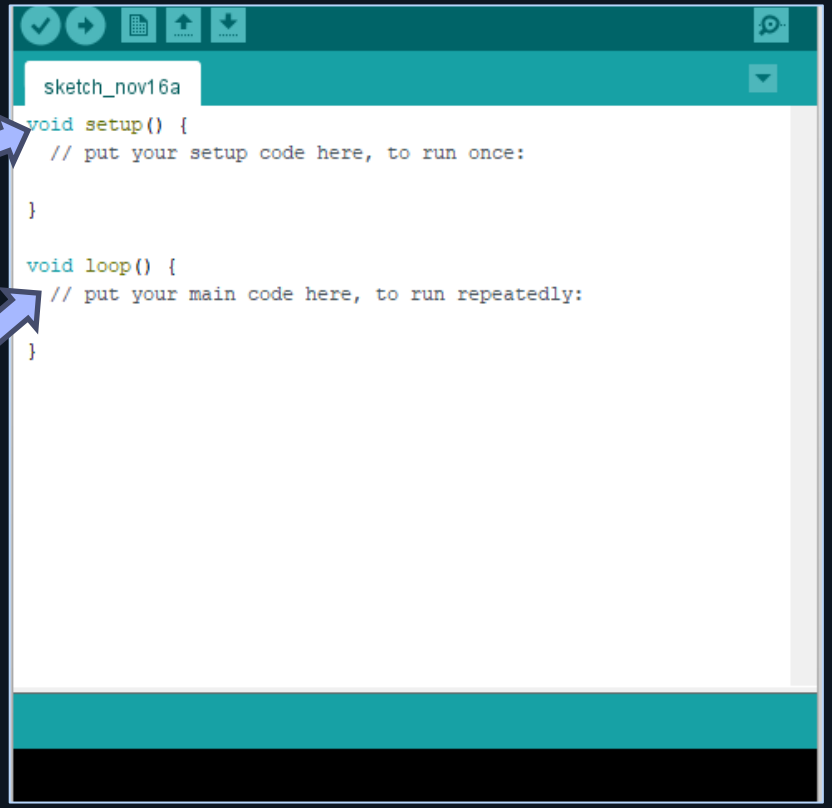


IDE.

Setup: Colocamos aqui tudo que será executado apenas 1x, além de declarar o que cada GPIO é exemplo saída ou entrada.

Aqui vão todos os códigos que irão rodar em loop.

Essa estrutura básica de programação sempre deve conter um void setup() e um void loop()

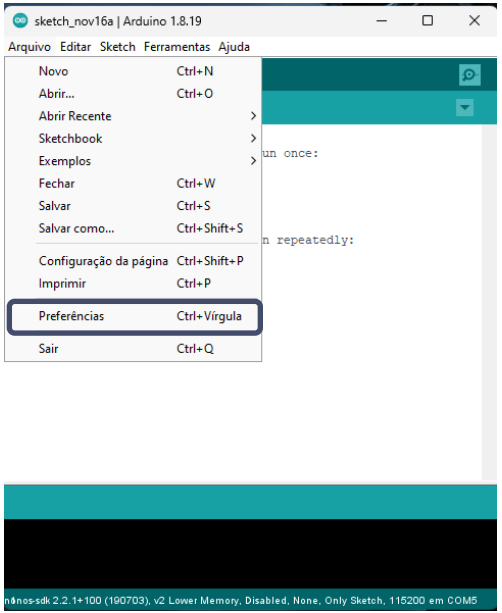


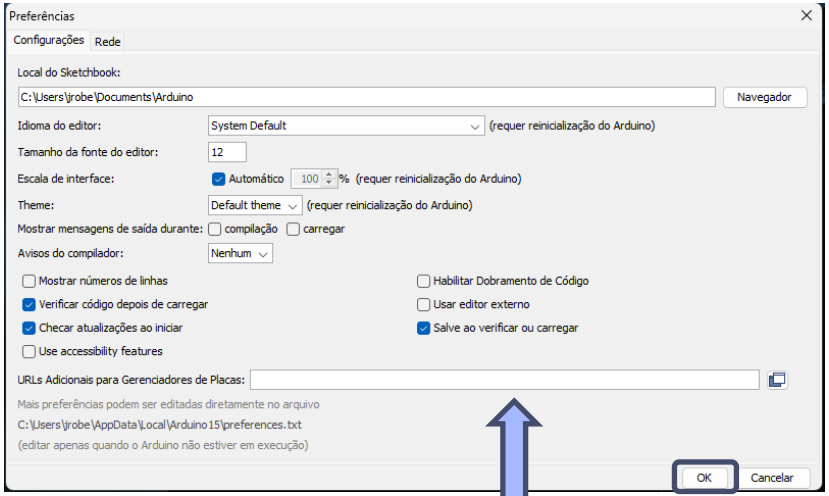
Diferenças entre Microcontrolador e Computador

Tabela comparativa

Tipo	Tamanho	Intervalo	Exemplo
int	32 bits	-2.147.483.648 a 2.147.483.647	int x = 10;
float	32 bits	+-3.402.235 ^{38}	float y = 3.14;
char	8 bits	-128 à 127	char c = ‘A’;
bool	8 bits	true ou false	bool estado = true;
String	Dinâmico	Cadeia de caracteres	String msg = “Oi”;

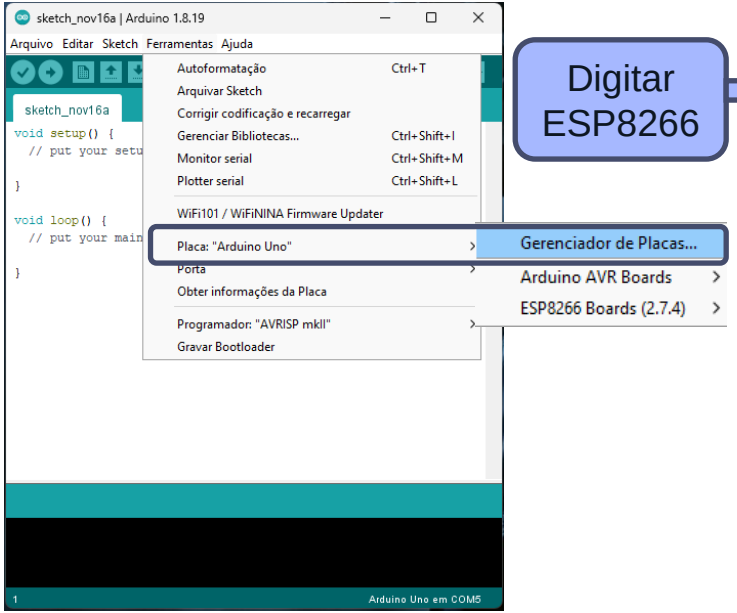
Nota: O ESP32 suporta tipos de dados maiores e mais modernos do que microcontroladores mais simples, como o Arduino Uno.



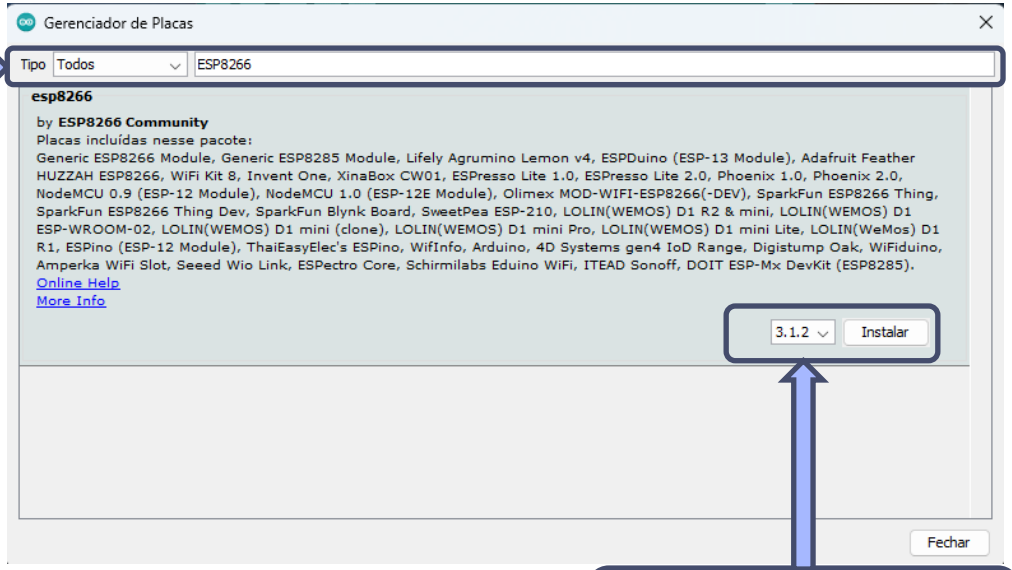


http://arduino.esp8266.com/stable/package_esp8266com_index.json

Configurar a IDE do Arduino para trabalhar com ESP8266

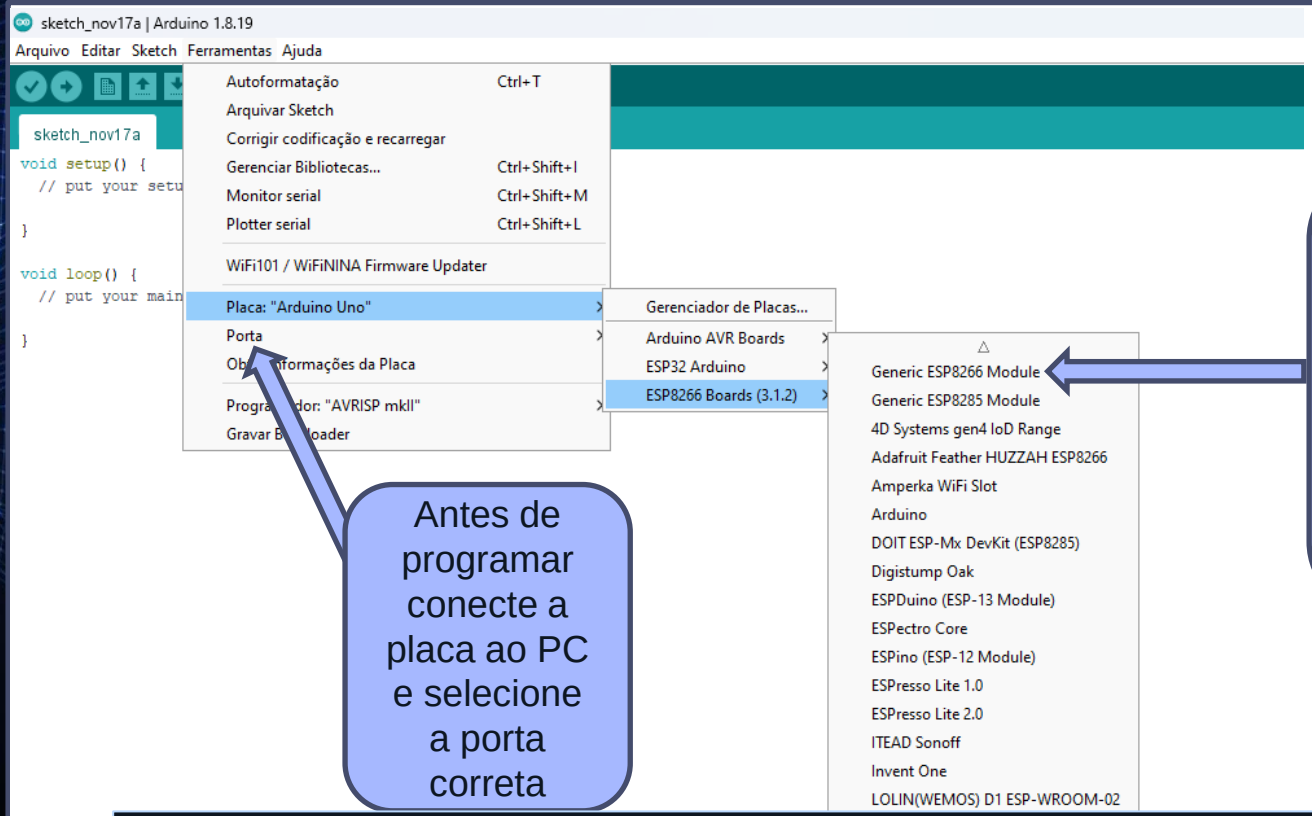


Digitar ESP8266



Procurar por ESP8266 e clicar em Instalar

Configurar a IDE do Arduino para trabalhar com ESP8266



Antes de programar conecte a placa ao PC e selecione a porta correta

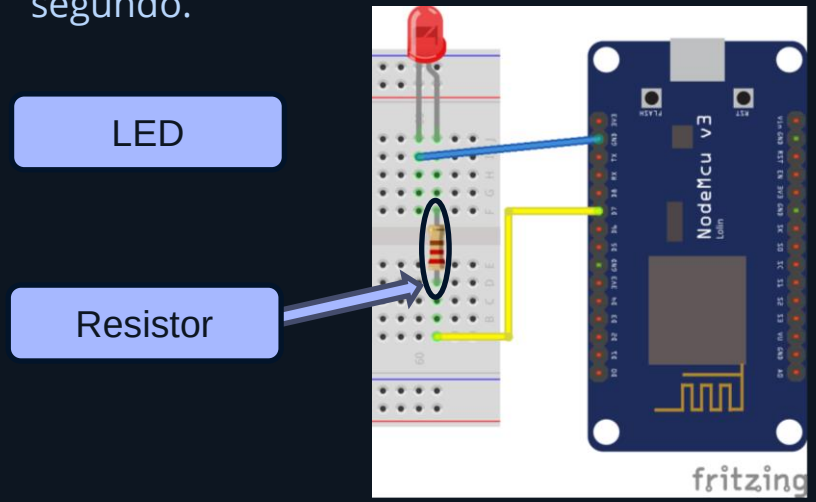
A placa deve ser selecionada em: Ferramentas; Placa; ESP8266 Board; Generic ESP8266 Module

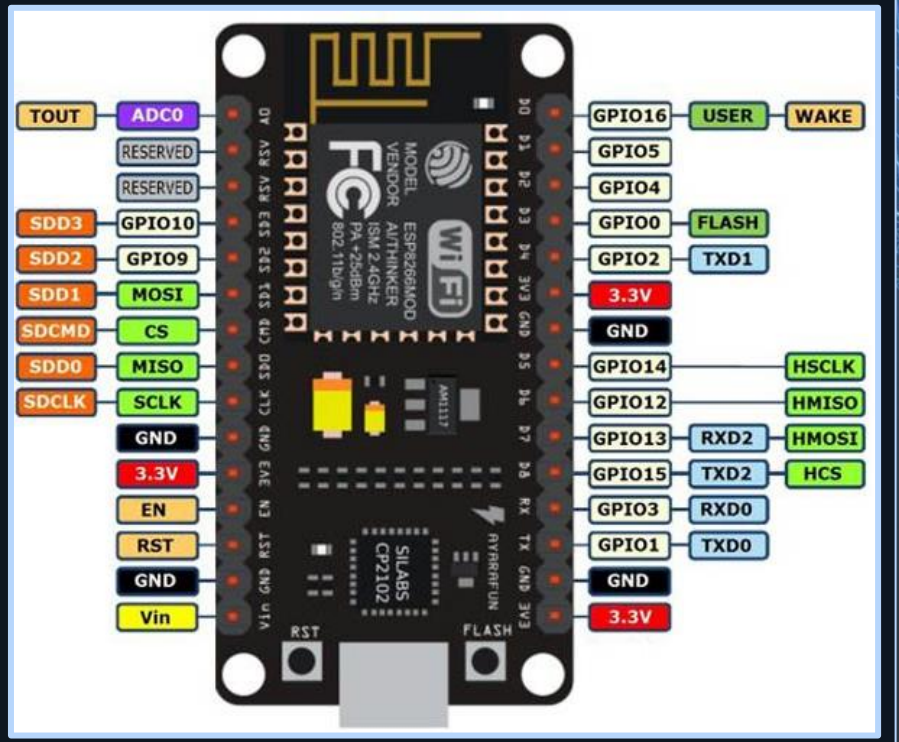
Configurar a IDE do Arduino para trabalhar com ESP8266

1º Programa

Vamos fazer o Hello Word do microcontrolador.

Vamos piscar um LED de 1segundo em 1 segundo.





Resistor – eletrônica básica

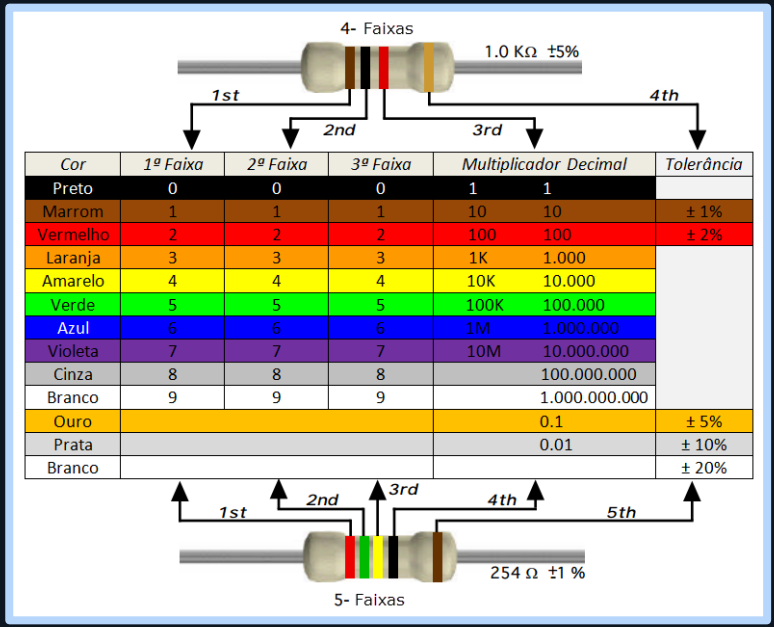
O resistor é um componente eletrônico que reduz, controla ou limita o fluxo de corrente elétrica em um circuito.
Representado por R e medido em Ohms (Ω).

Principais Funções:

- Proteger componentes sensíveis (ex.: LEDs) contra excesso de corrente.
- Ajustar níveis de tensão em circuitos.
- Controlar a intensidade de sinais elétricos.

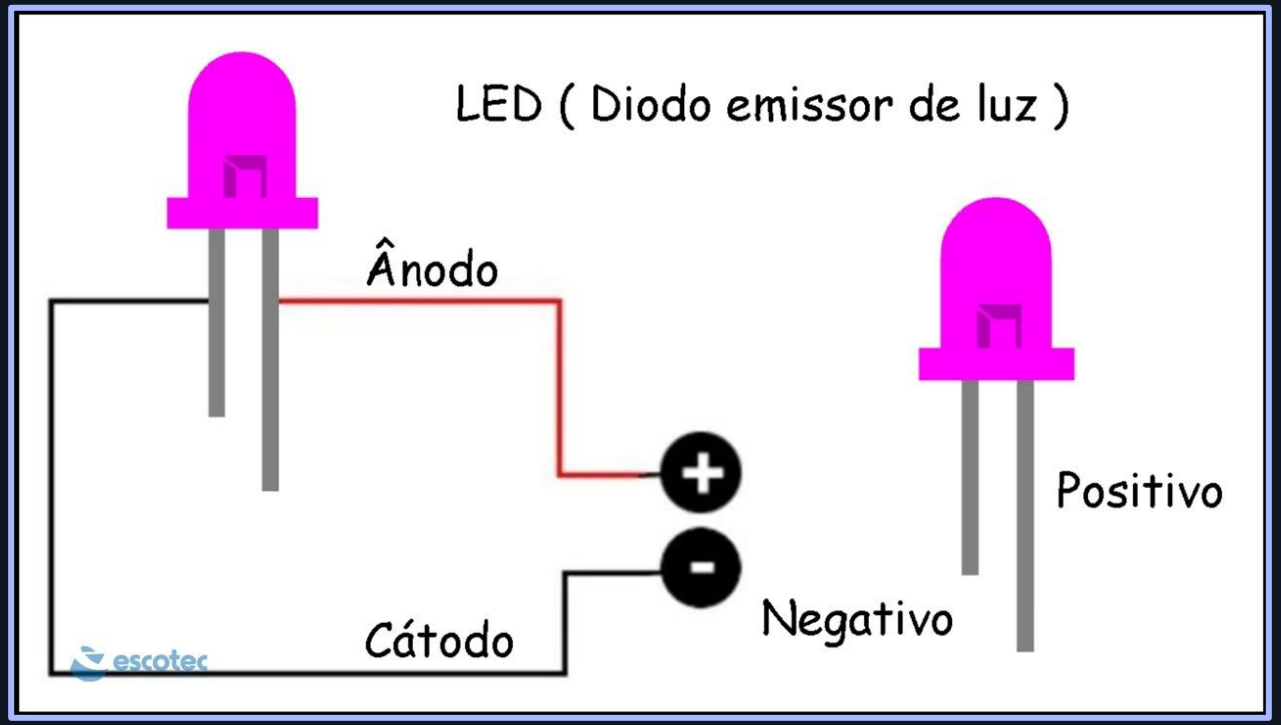
Proteção: Evita que componentes sejam danificados por excesso de corrente.
Estabilidade: Garante o funcionamento correto do circuito.

Exemplo Prático:
Um LED sem resistor pode queimar porque ele não suporta altas correntes diretamente.



Cor	1ª Faixa	2ª Faixa	3ª Faixa	Multiplicador Decimal		Tolerância
Preto	0	0	0	1	1	
Marrom	1	1	1	10	10	± 1%
Vermelho	2	2	2	100	100	± 2%
Laranja	3	3	3	1K	1.000	
Amarelo	4	4	4	10K	10.000	
Verde	5	5	5	100K	100.000	
Azul	6	6	6	1M	1.000.000	
Violeta	7	7	7	10M	10.000.000	
Cinza	8	8	8		100.000.000	
Branco	9	9	9		1.000.000.000	
Ouro					0.1	± 5%
Prata					0.01	± 10%
Branco						± 20%

Ânodo e Cátodo



1º Programa - Solução

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Número da GPIO, D7 = GPIO13 → Pino onde está ligado o LED

OUTPUT → Saída do bit;
INPUT → Entrada do bit;

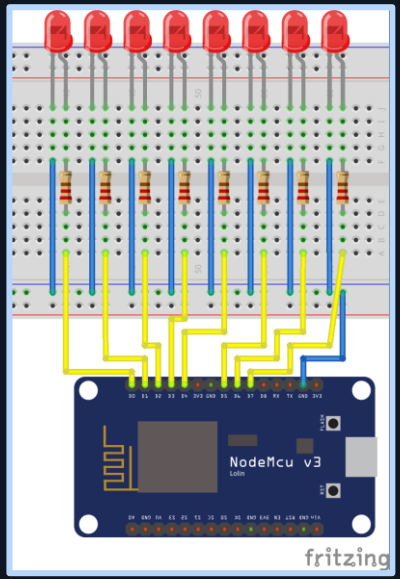
digitalWrite → Função para controlar uma porta digital de saída

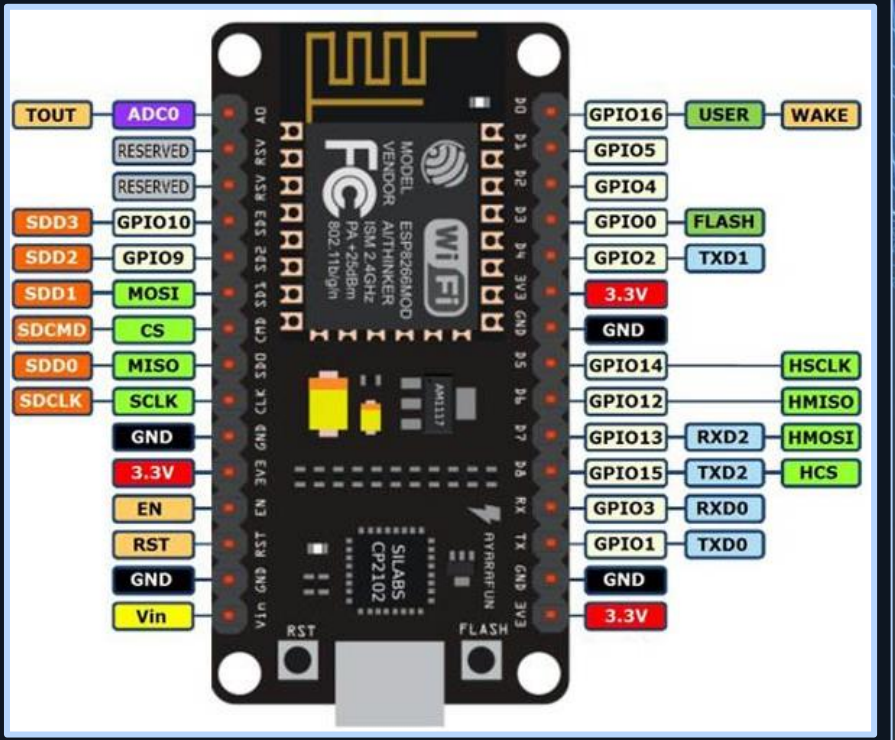
HIGH e LOW → Comandos de Ligar e Desligar, HIGH = 1 e LOW = 0

delay(valor) → tempo de espera do microcontrolador em milisegundos

2º Programa

Façam um programa contendo 8 LEDs onde eles devem ligar em sequência e desligar na sequência inversa.

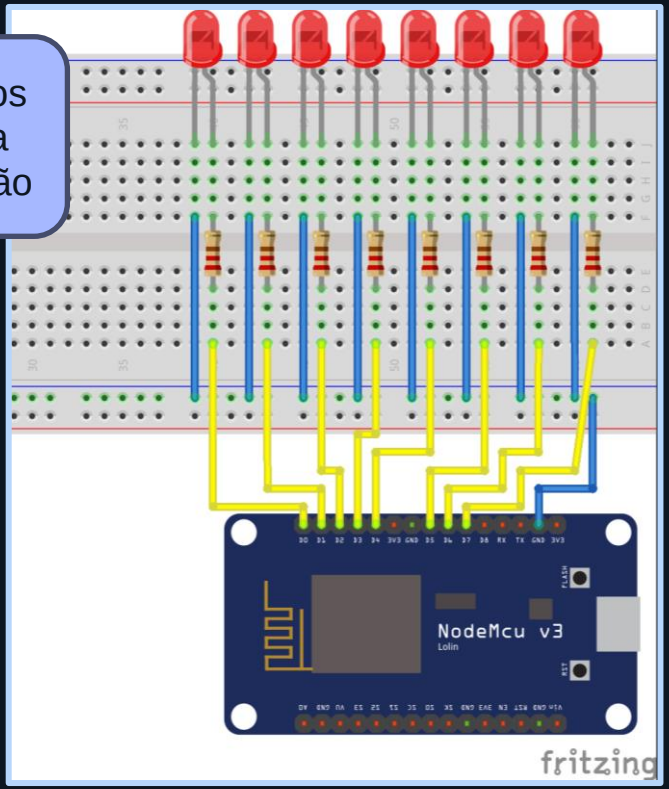




2º Programa - Solução

```
int ledPins[] = {2, 4, 5, 18, 19, 21, 22, 23};
int numLeds = sizeof(ledPins) / sizeof(ledPins[0]);

void setup() {
for (int i = 0; i < numLeds; i++) {
    pinMode(ledPins[i], OUTPUT);
  }
}

void loop() {
  for (int liga = 0; liga < numLeds; liga++) {
      digitalWrite(ledPins[liga], HIGH);
      delay(500);
    }

    for (int desliga = numLed-1; desliga >=0; desliga--) {
      digitalWrite(ledPins[desliga], LOW);
      delay(500);
    }
}
```

Colocar os pinos da sua ligação



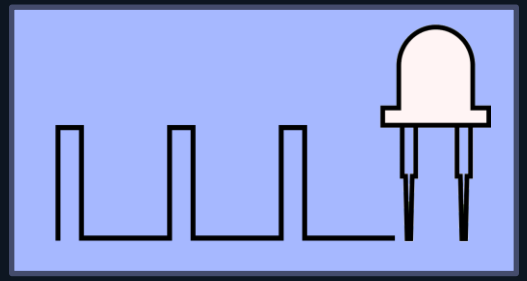
Saídas Analógica

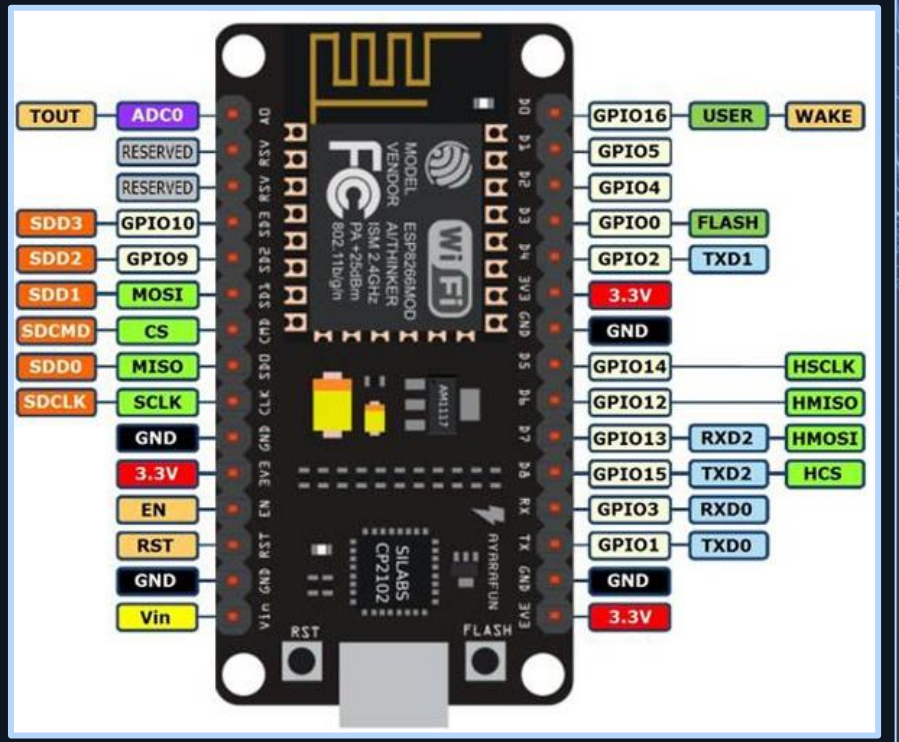
Introdução à Saída Analógica com PWM

A saída analógica no ESP8266 é simulada por meio de PWM (Pulse Width Modulation), que cria uma variação média de tensão controlando o tempo de alta e baixa em um sinal digital.

Destaques:

- Utilizada para controlar brilho de LEDs, velocidade de motores, entre outros.
- O ESP8266 tem resolução de 10 bits



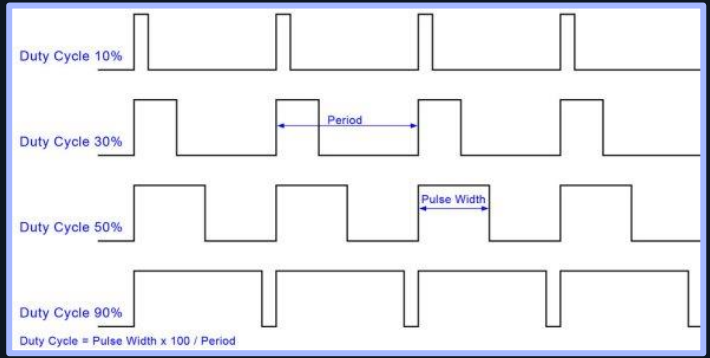


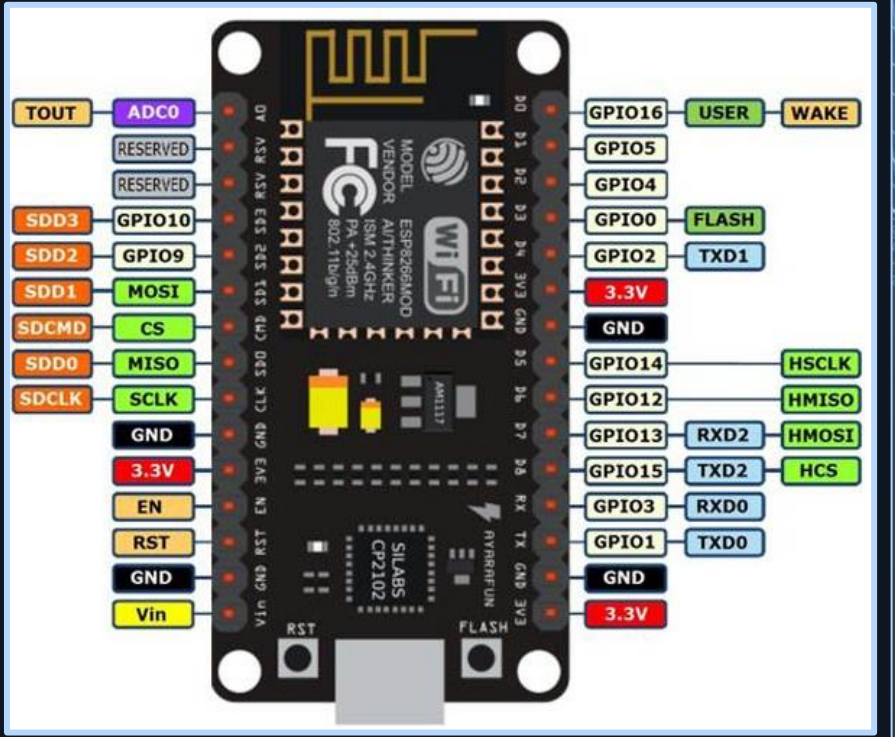
O que é PWM

PWM é uma técnica que varia o ciclo de trabalho de um sinal digital para simular uma saída analógica.

Conceitos:

- **Duty Cycle:** Porcentagem do tempo que o sinal permanece em alta (HIGH).
- **Frequência:** Quantas vezes o ciclo (HIGH + LOW) se repete por segundo.





PWM ESP8266

Podemos utilizar qualquer pino como “saída analógica”.

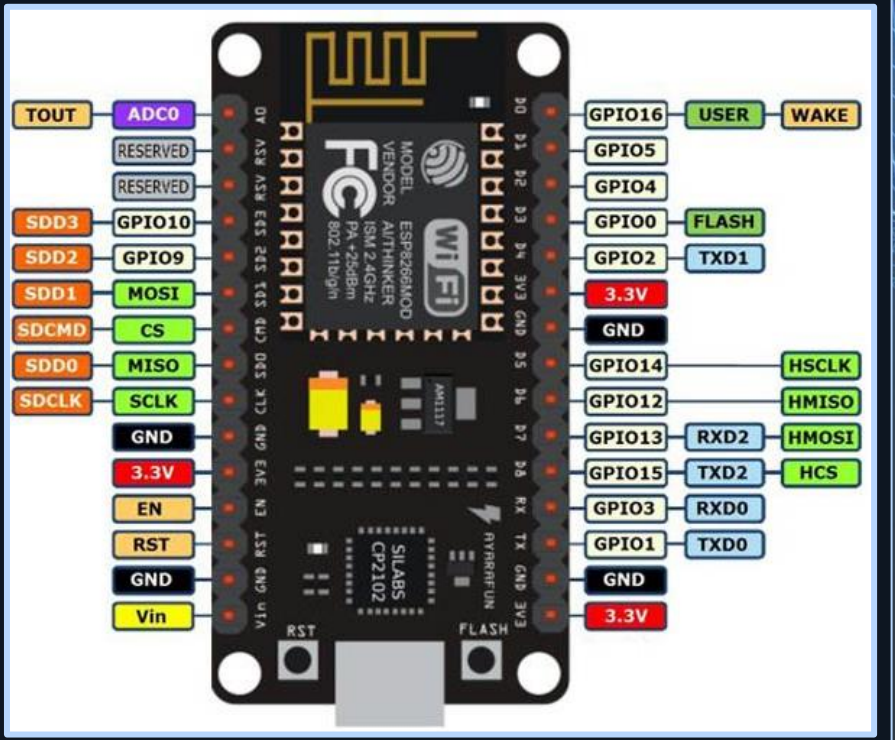
Para que funcione perfeitamente temos 2 passos de configuração:

- No SETUP temos que configurar o pinMode(pino, OUTPUT);
- No LOOP temos o analogWrite(pino, valorPWM);

Exemplo:

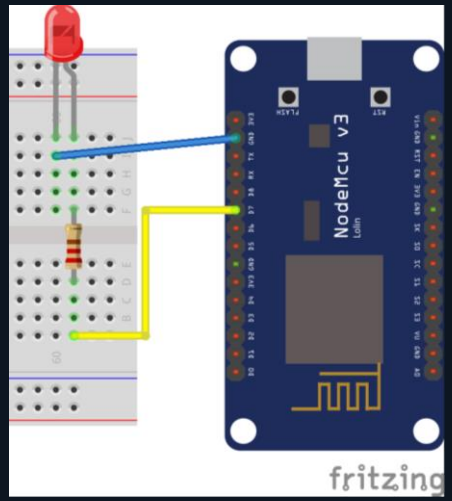
- analogWrite(2, 255) → 255 – LED Totalmente ligado;
- analogWrite(2, 0) → 0 – LED totalmente desligado.

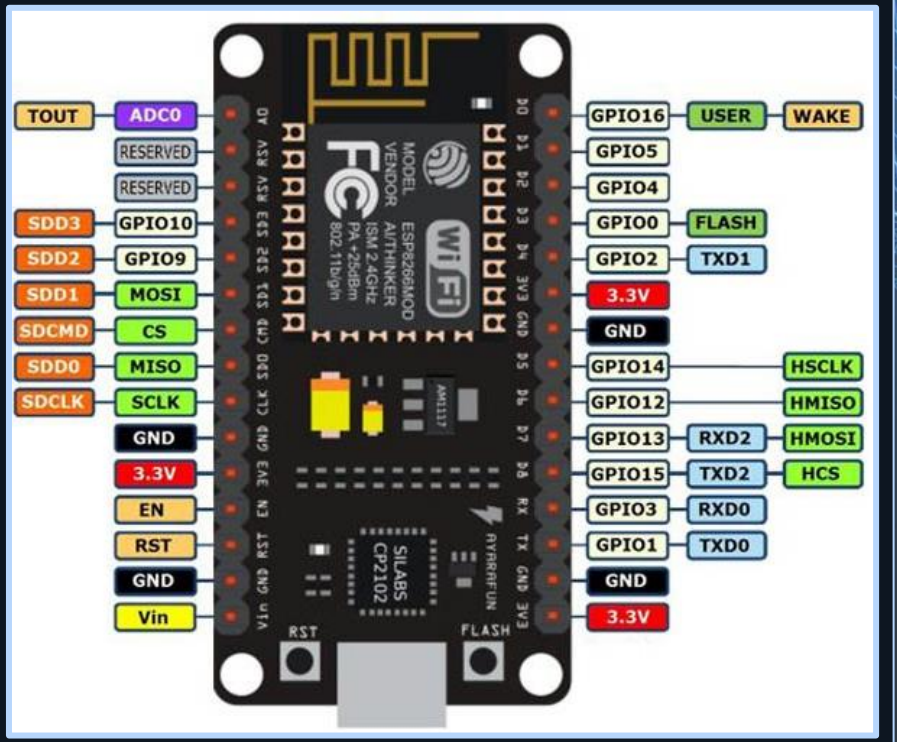
- Caso eu queira que o LED fique com 50% do tempo ligado o valor é 127.



1º Programa - PWM

Vamos criar um código para que o LED lige gradativamente e desligue gradativamente





1º Programa - PWM

```
#define led 13
void setup() {
  pinMode(led, OUTPUT);

}

void loop() {
  for(int i = 0; i<=255; i++){
    analogWrite(led,i);
    delay(5);
  }
  delay(1000);
  for(int i = 255; i>=0; i--){
    analogWrite(led,i);
    delay(5);
  }
  delay(1000);
}
```

#define → Forma de falar para o programa usar o valor 13 quando encontra a palavra led

OUTPUT → Saída do bit;
INPUT → Entrada do bit;

Ligando

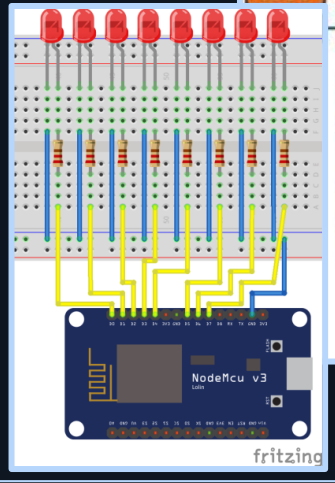
Desligando

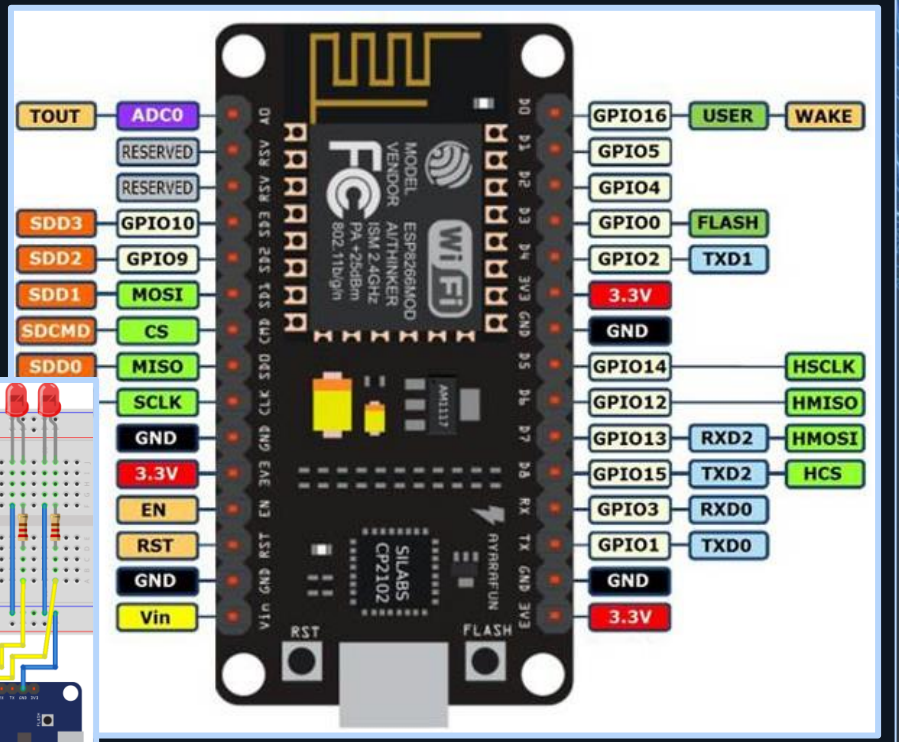
analogWrite(pino, PWM) → Faz a variação do LED ligando e desligando.

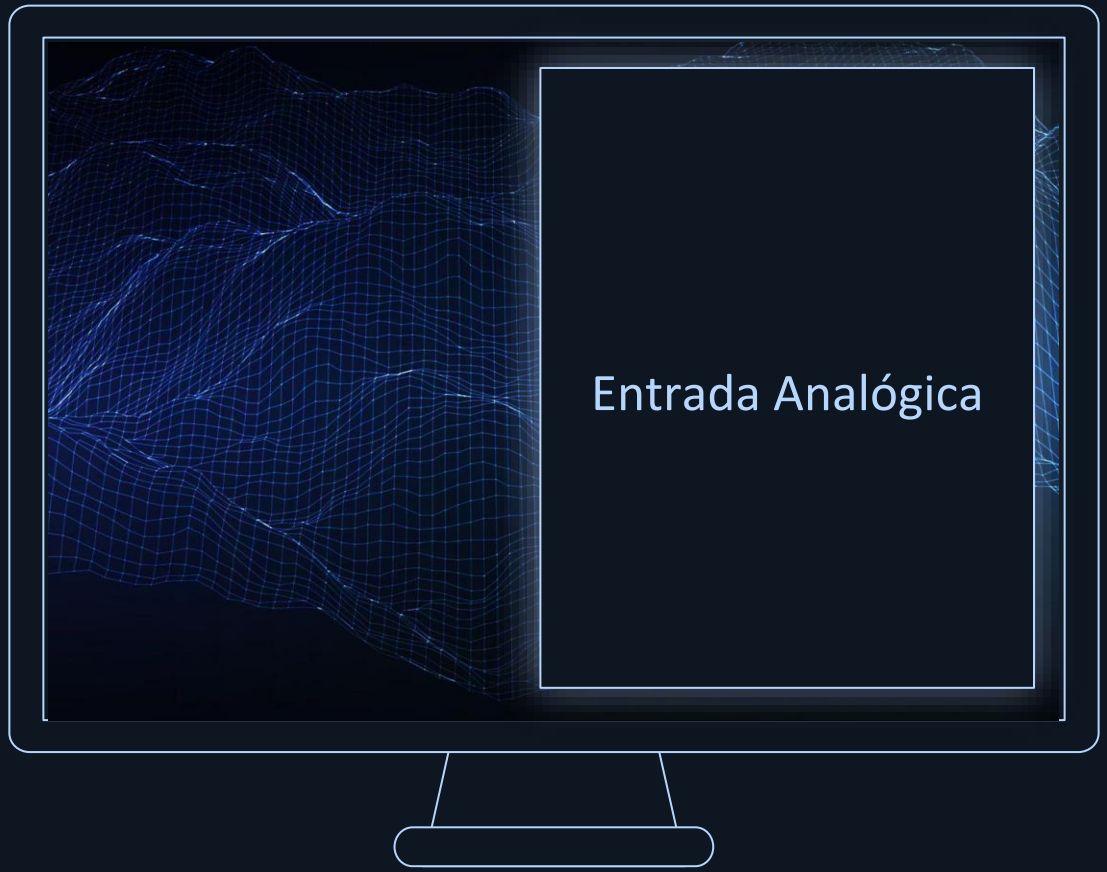
2º Programa - PWM

Crie um programa para o ESP8266 que controle 8 LEDs usando PWM. O programa deve:

- Ligar cada LED de forma gradual (aumentando o brilho de 0% a 100%) um por vez.
- Depois que todos os LEDs estiverem acesos, desligar cada LED de forma gradual (reduzindo o brilho de 100% a 0%), um por vez.
- Repetir o processo continuamente.







Entrada Analógica

ESP8266 – Entrada Analógica

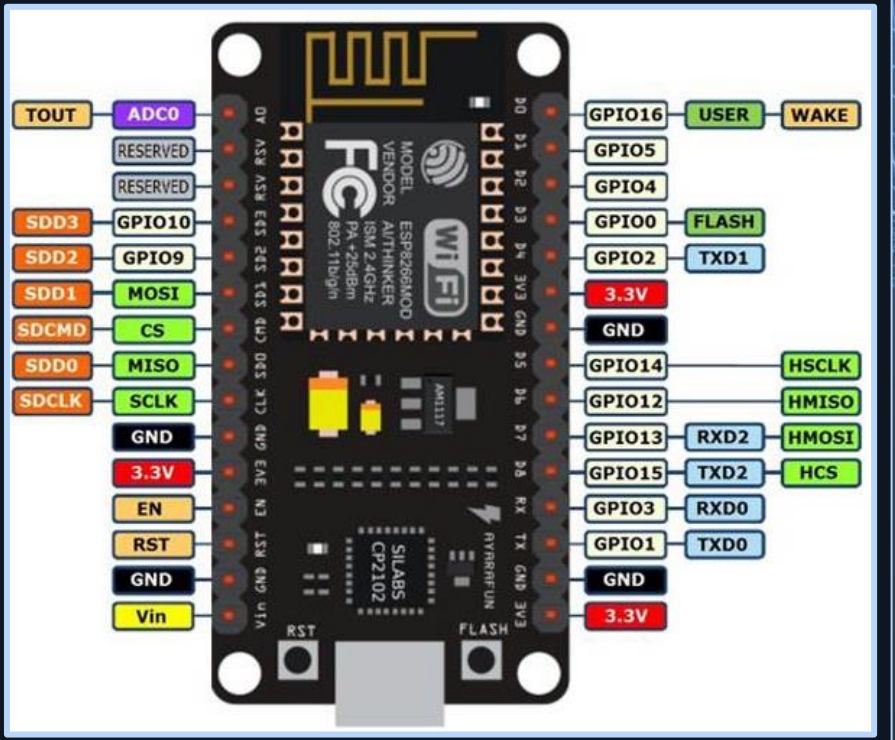
Uma entrada analógica é usada para ler sinais contínuos (como a tensão variável de um sensor) e convertê-los em valores digitais.

- Usada para medir sensores de luz, temperatura, umidade, etc.

- Representa variações de tensão como valores numéricos.

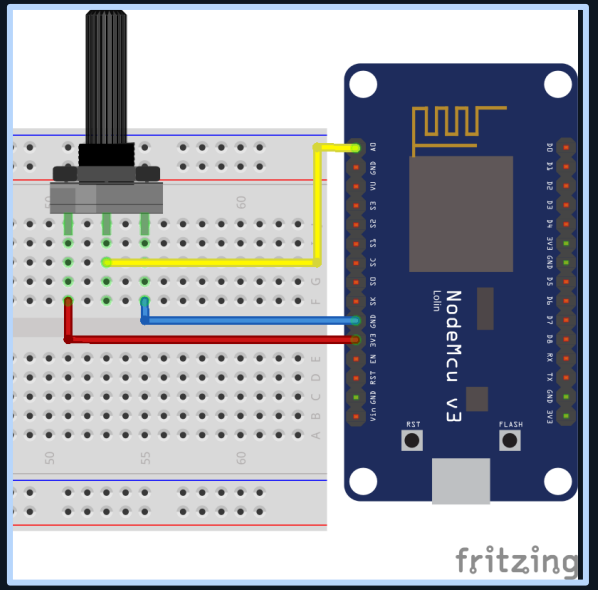
Características:
- Apenas 1 canal ADC (ADC0).
- Resolução: 10 bits (valores de 0 a 1023).
- Intervalo de tensão: 0V a 1V.

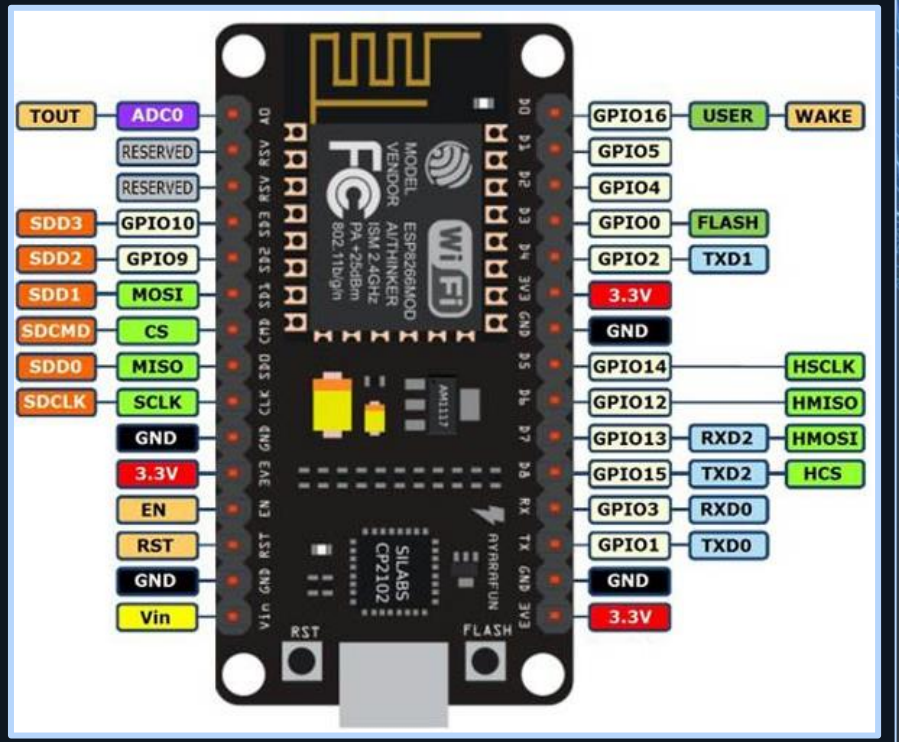
Atenção: Tensões acima de 1V precisam de um divisor de tensão para proteção.



1º Programa - Analógico

Vamos criar um código para mostrar o valor de um potenciômetro no Serial Monitor



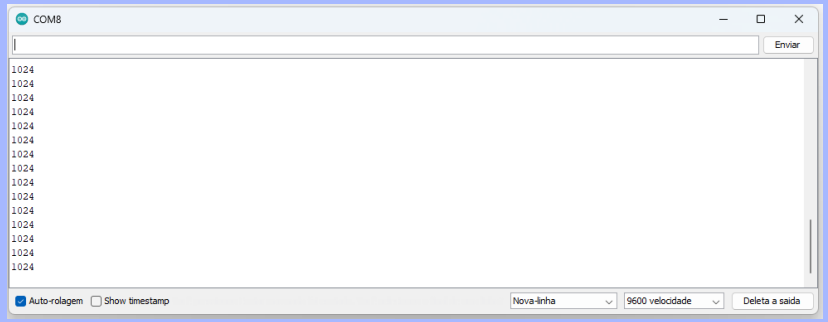


1º Programa - Analógico

```
void setup() {
  Serial.begin(9600);

}

void loop() {
  int potenciometro = analogRead(A0);
  Serial.println(potenciometro);
  delay(1000);
}
```

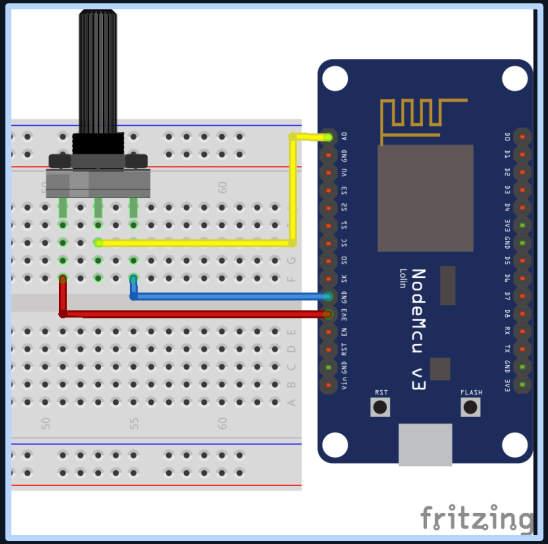
Serial.begin(9600) → Monitor Serial, ele mostra a atualização do que acontece na USB.

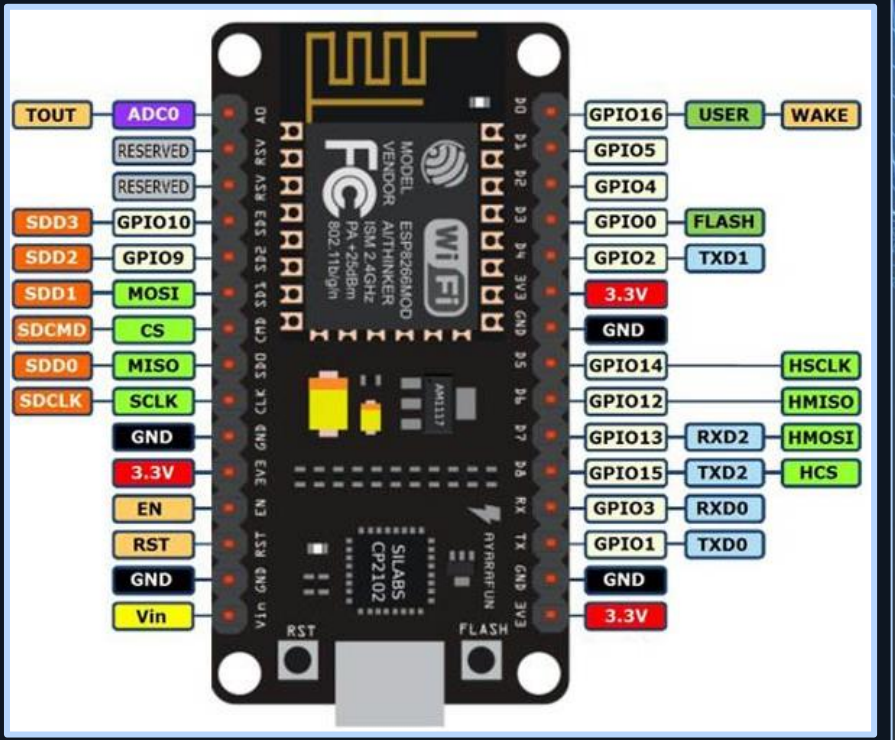
analogRead(A0) → comando para leitura do sinal analógico



2º Programa - Analógico

Vamos criar um código para mostrar o valor de um potenciômetro em porcentagem no Serial Monitor

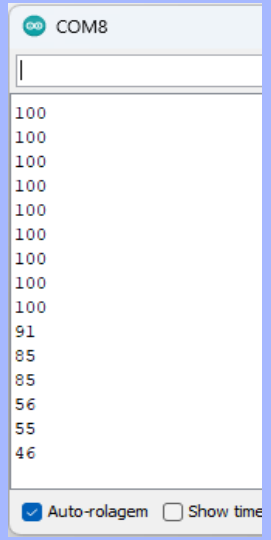




2º Programa - Analógico

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int potenciometro = analogRead(A0);
  int porcentagem = map(potenciometro,0,1024,0,100);
  Serial.println(porcentagem);
  delay(1000);
}
```

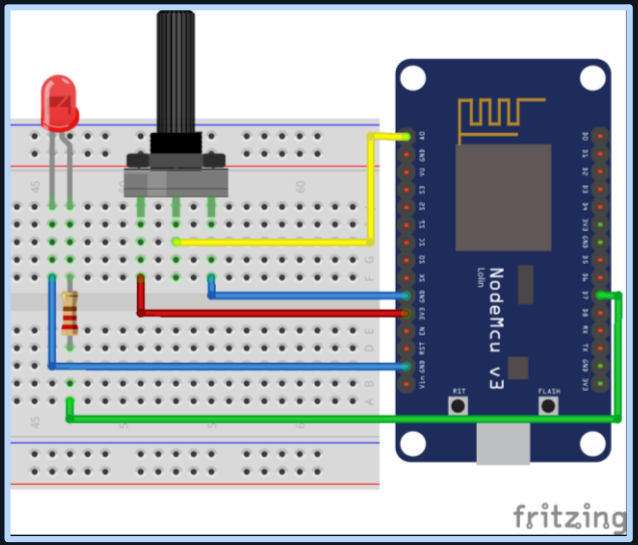
Saída no Serial Monitor

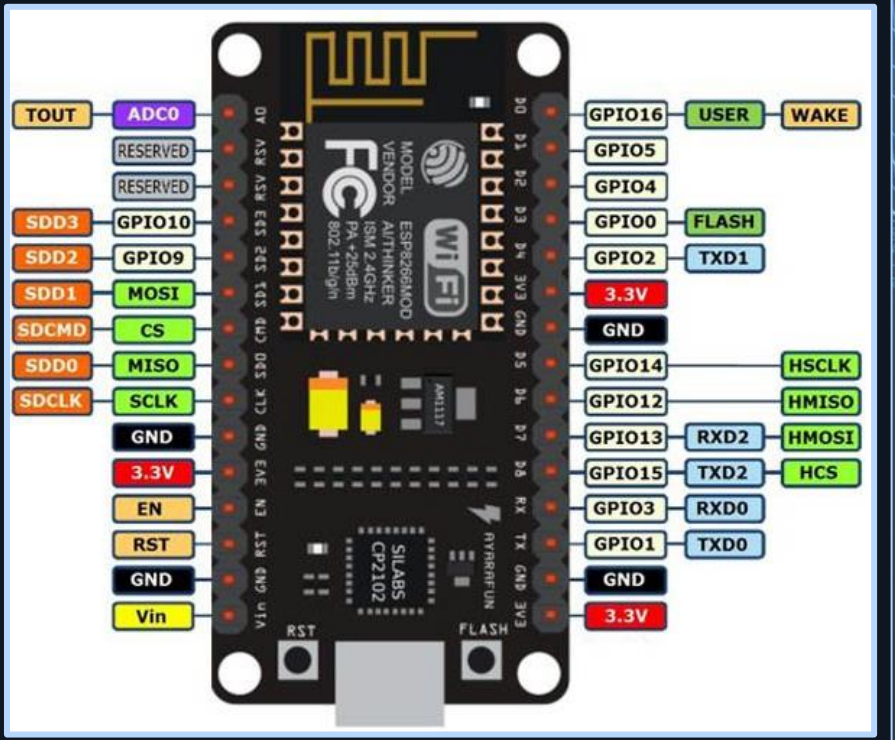


Função map→ faz o mapeamento de um valor e converte em outro fazendo uma regra de 3.
Comando:
map(Valor-Entrada;ValorInicialAntigo;ValorFinalAntigo;ValorInicialNovo;ValorFinalNovo);

3º Programa - Analógico

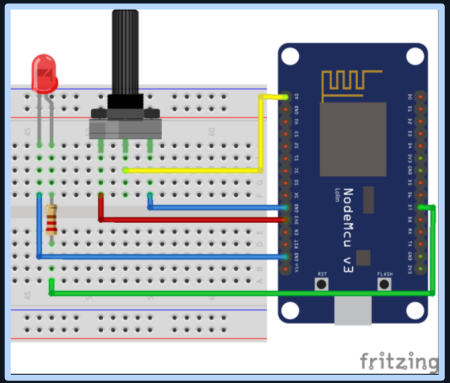
Ligue um LED utilizando o Potenciômetro a medida que gira ele a intensidade do LED aumenta ou diminui.

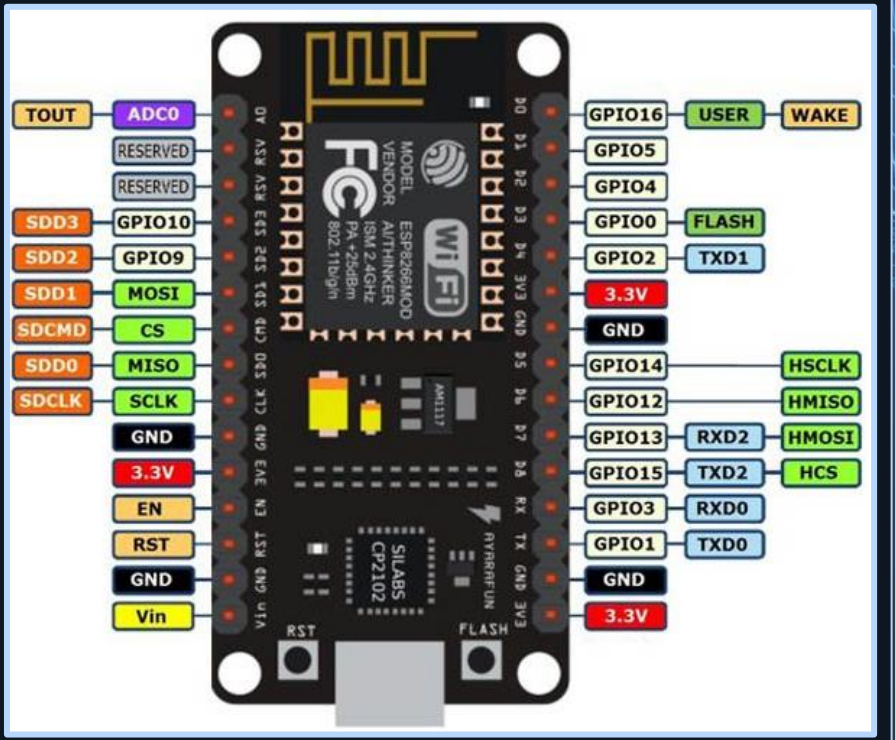




3º Programa - Analógico

```
#define led 13

void setup() {
  pinMode(led,OUTPUT);
}

void loop() {
  analogWrite(led,map(analogRead(A0),0,1024,0,255));
}
```





Entrada Digital

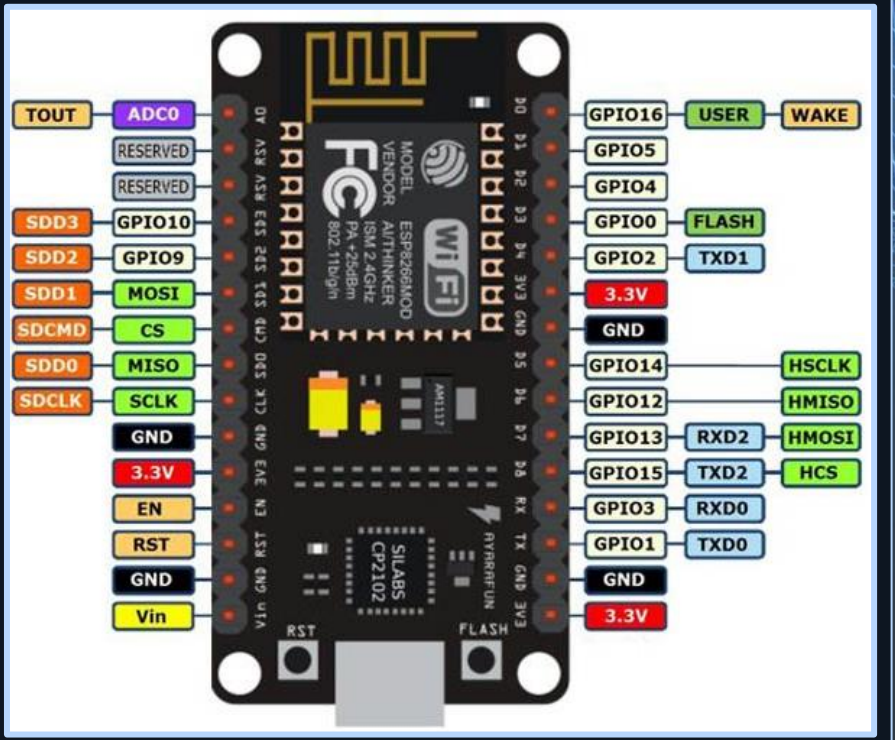
ESP8266 – Entrada Digital

Uma entrada digital é usada para detectar o estado de um sinal: LIGADO ou DESLIGADO (nível lógico 1 ou 0).

Valores possíveis: HIGH (1) ou LOW (0).
Conectada a dispositivos como botões, chaves ou sensores digitais.

Exemplo:
- Botão pressionado: HIGH.
- Botão solto: LOW.

Definir pinMode(pino, INPUT);
Ler botão: digitalRead(pino);



ESP8266 – Entrada Digital

Onde Usamos Entradas Digitais?

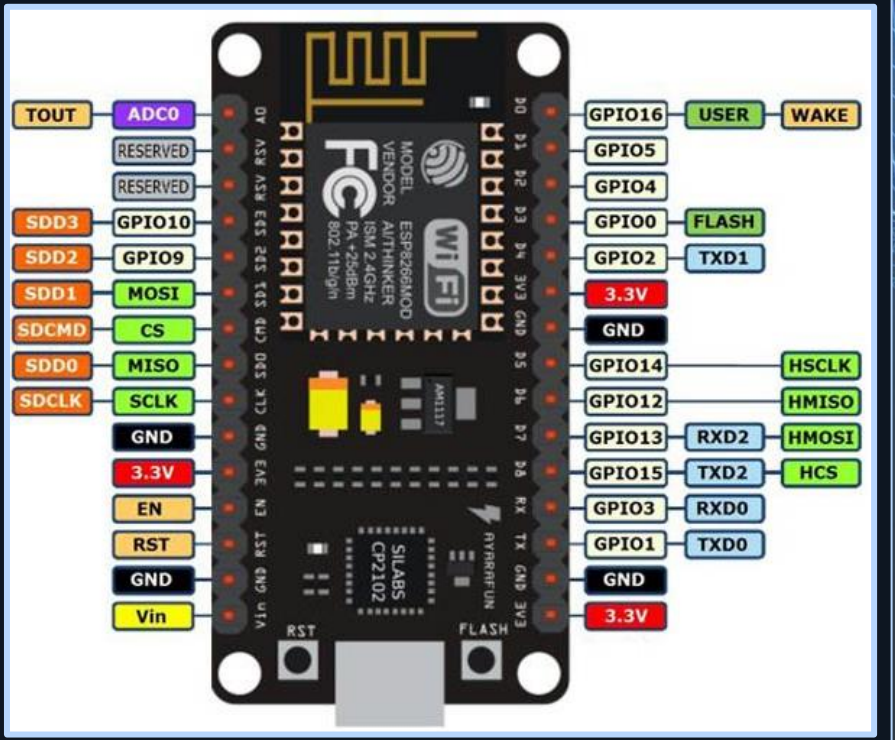
Detectar estados de:

- Botões e chaves.
- Sensores digitais (por exemplo, sensor infravermelho ou ultrassônico com saída digital).

Controlar lógica em projetos IoT:

Acionar dispositivos apenas quando uma condição for atendida.

É necessário utilizar resistores para fazer botões Pull-up e Pull-down



Pull-up e Pull-down

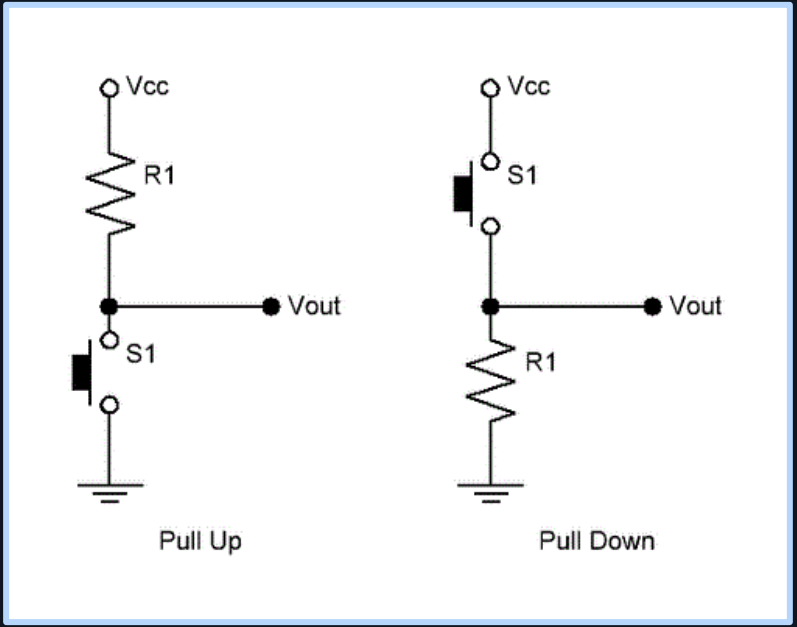
Quando um botão ou sensor não está conectado diretamente a VCC ou GND, o pino digital pode ficar em um estado "flutuante", gerando leituras instáveis (ruídos).

Pull-up → sempre manda 1 no pino quando pressionamos o botão ele manda 0;

Pull-down → sempre manda 0 no pino quando pressionamos o botão ele manda 1;

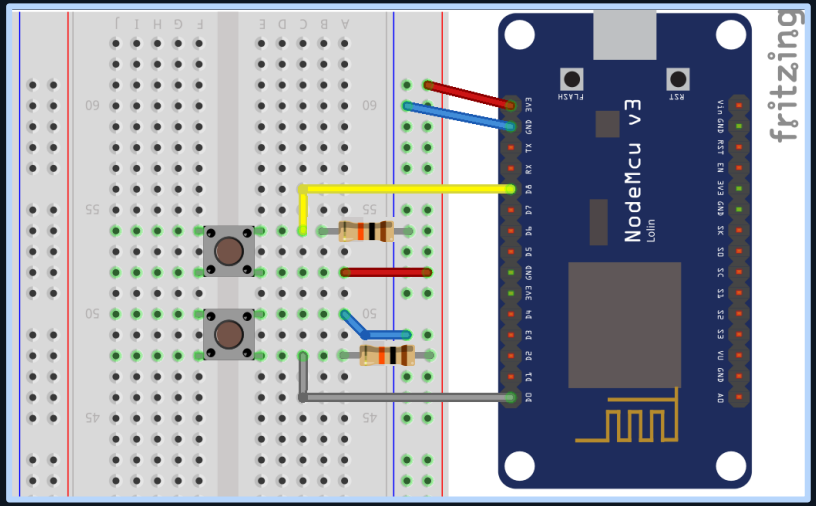
R1 valor 10KΩ

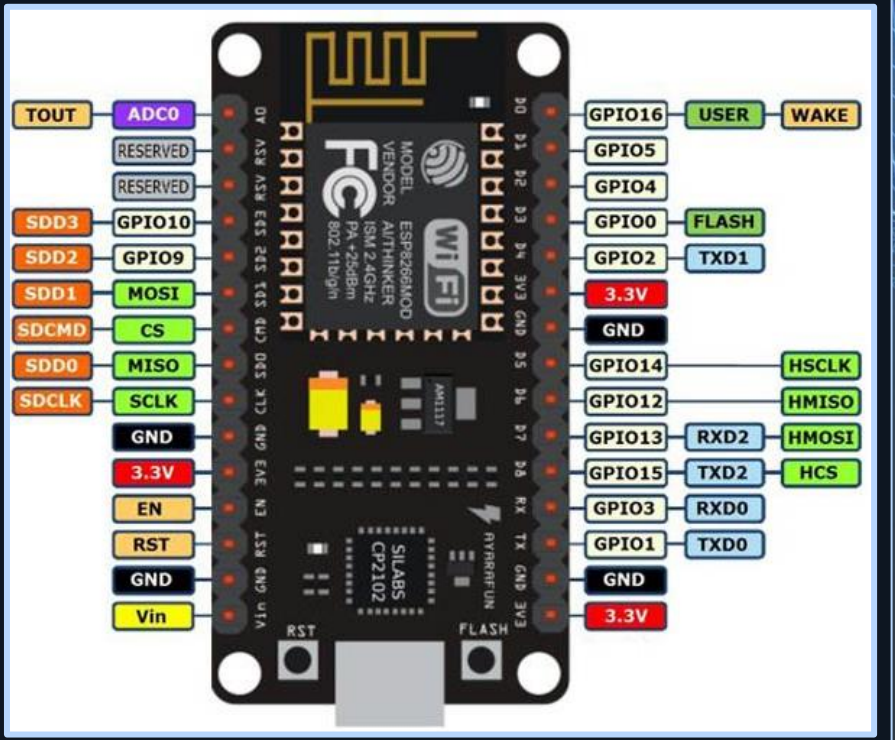
Obs: Não utilizar botão sem ser pull-up ou pull-down.



1º Programa – Entrada Digital

Faça um programa para mostrar o estado do botão no serial monitor.
Faça um botão por vez.





1º Programa – Entrada Digital

```
#define btn1 15
#define btn2 16

void setup() {
  pinMode(btn1,INPUT);
  pinMode(btn2,INPUT);
  Serial.begin(9600);
}

void loop() {
  Serial.println(digitalRead(btn1));
  //Serial.println(digitalRead(btn2));
}
```

pinMode(pino, INPUT); → declara o pino como entrada

digitalRead(pino) → Lê o estado (1 ou 0) do botão.
// → comentário

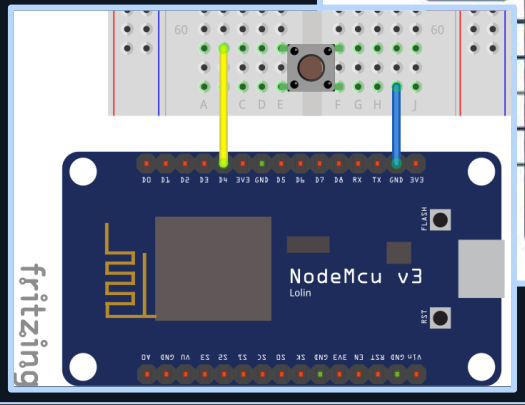
Entrada Digital – Pull up e Pull down

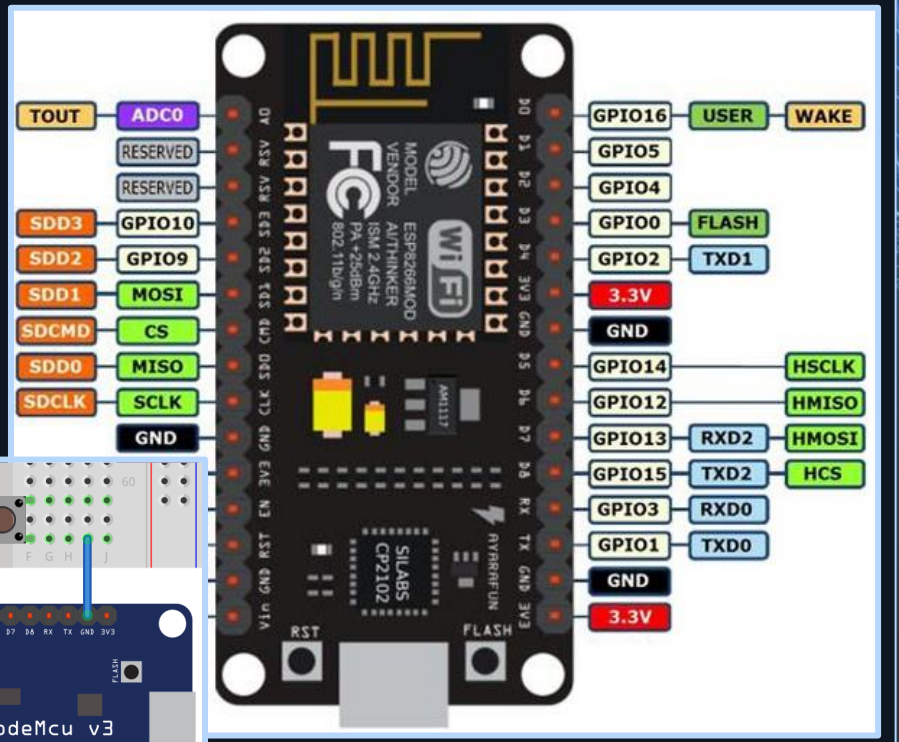
Caso não tenhamos nenhum recurso extra para colocar um botão no microcontrolador, podemos utilizar o pull-up interno do microcontrolador.

O microcontrolador vem com um resistor ligado em seus pinos digitais o que torna possível ligar o botão diretamente nele sem a necessidade de um resisto.

Declarar como:

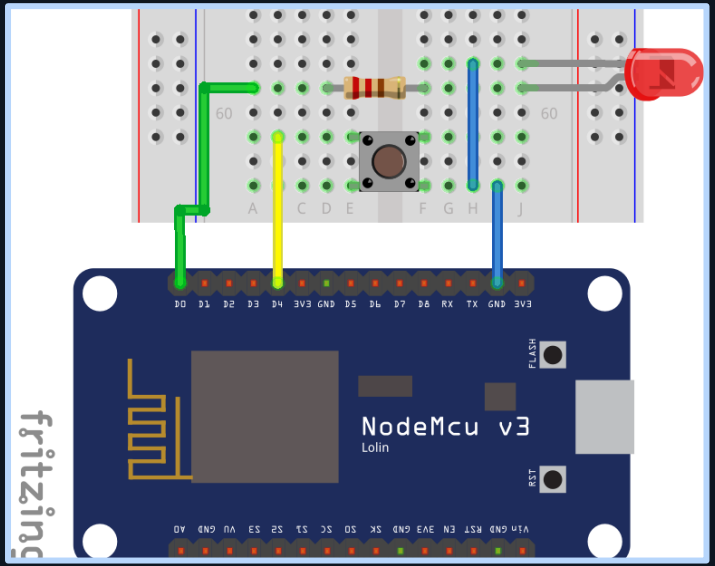
pinMode(pino,INPUT_PULLUP);

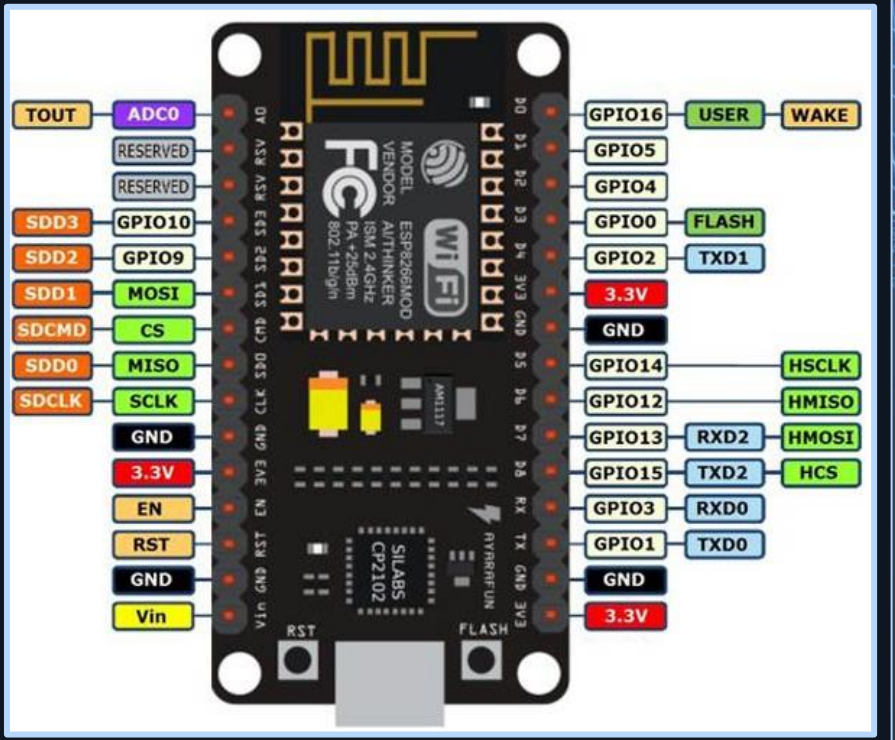




2º Programa – Entrada Digital

Faça um programa para quando pressionar o botão o LED ligue. **Não usar IF**

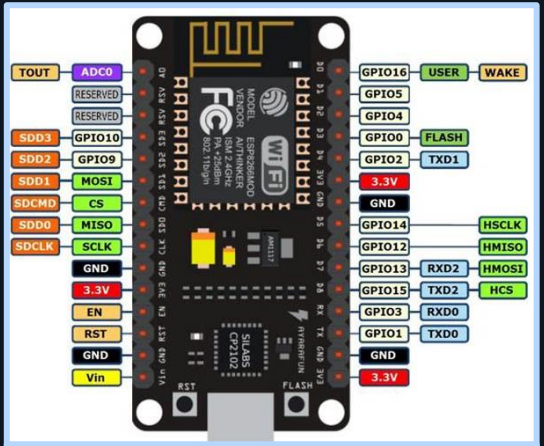


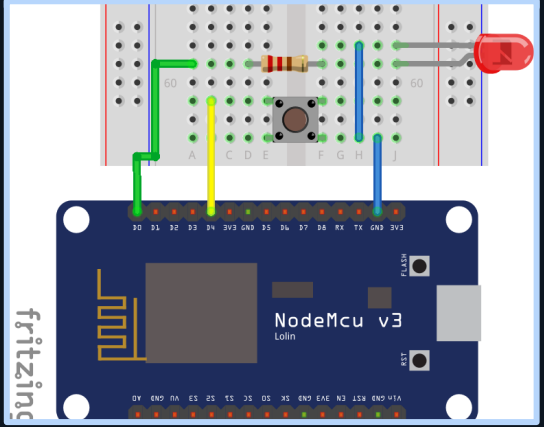


2º Programa – Entrada Digital

```
#define led 16
#define btn 2

void setup(){
  pinMode(led,OUTPUT);
  pinMode(btn,INPUT_PULLUP);
}

void loop(){

  digitalWrite(led,!digitalRead(btn));
}
```

Negar o estado do botão.
Estamos usando pull-up que sempre manda 1





Entrada Digital - Problema

Entrada Digital – problema

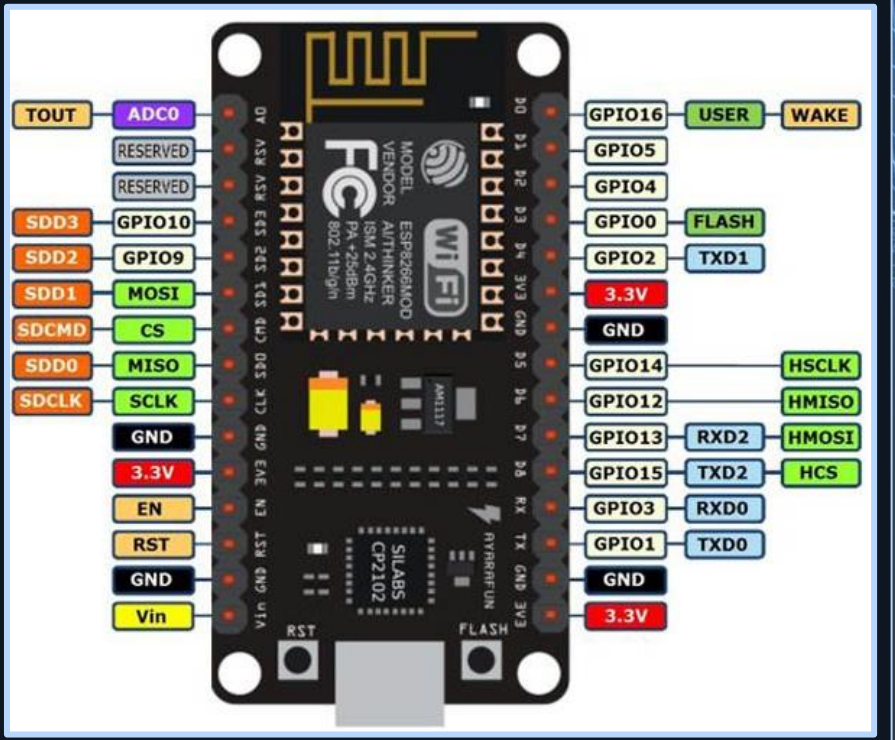
Entendendo o Problema de **Bounce**

Bounce ocorre quando um botão ou chave é acionado, causando oscilações rápidas no sinal elétrico antes de estabilizar.

Causa:
Contato mecânico imperfeito nos botões e chaves.

Problema:
O microcontrolador interpreta várias mudanças de estado indesejadas, gerando leituras incorretas.

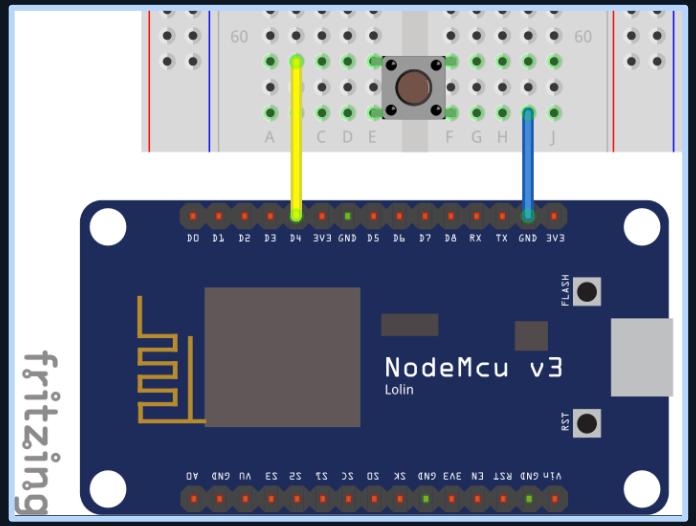
Exemplo:
Ao pressionar um botão, o microcontrolador pode ler várias transições HIGH e LOW antes do sinal estabilizar.

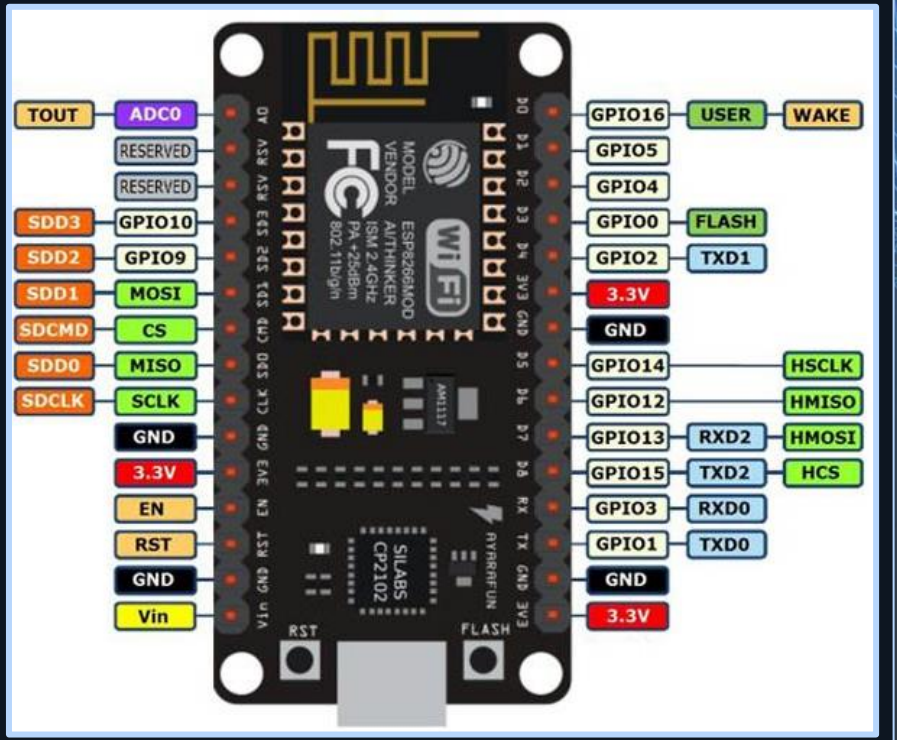


Entrada Digital – problema

Simulando o problema:

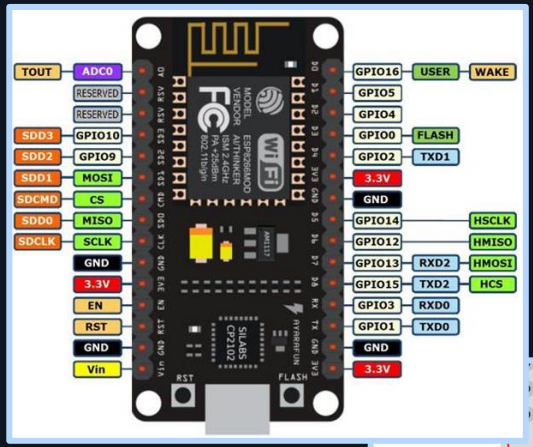
Vamos criar um programa de contagem de vezes que pressionamos um botão para ver o problema

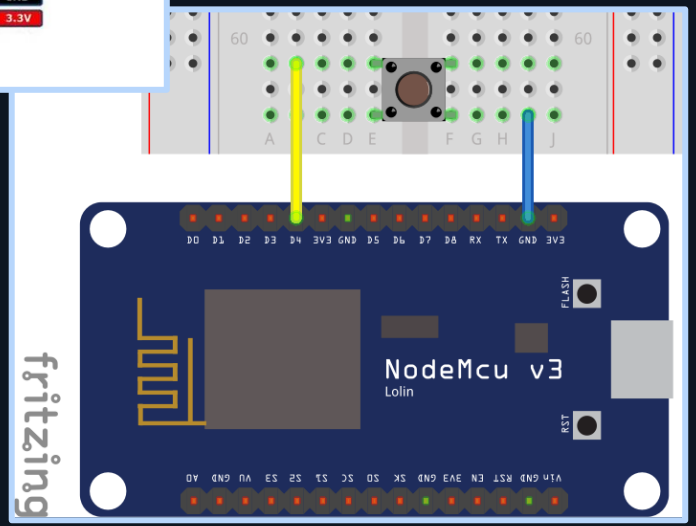




Entrada Digital – problema

```
#define btn 02
int contador;

void setup(){
  pinMode(btn,INPUT_PULLUP);
  Serial.begin(9600);
}

void loop(){
  if (digitalRead(btn) == 0){
    contador++;
  }
  Serial.println(contador);
}
```





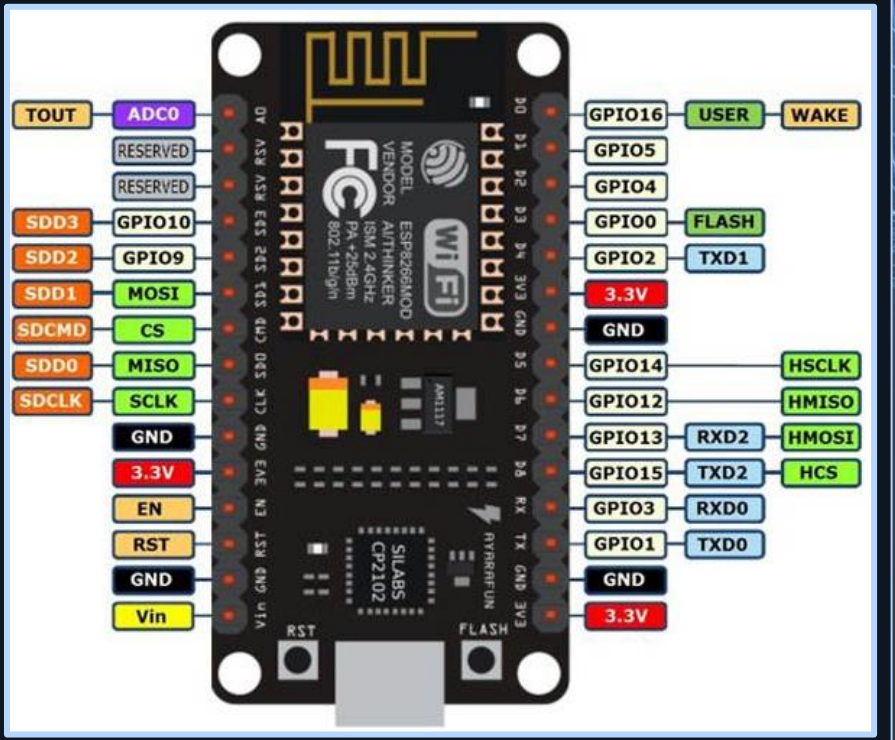
Repare que ao pressionar o botão apenas uma vez a variável aumenta muito rápido.

Entrada Digital – Resolvendo bounce

Para solucionar esse problema temos uma solução chamada Debounce é relativamente simples a implementação, não requer nenhum equipamento diferente só precisamos acertar o código.

Para isso vamos utilizar variáveis diferentes e utilizar um controle de tempo mais fino chamado millis().

O millis trabalha um pouco diferente do delay, o delay causa uma “pausa” no microcontrolador, já o millis só executa quando o tempo interno do microcontrolador atingir uma determinada condições vamos ver a solução com o millis.



Entrada Digital – problema

```
#define btn 02
int contador;
unsigned long ultimapressao;
int tempo = 100;

void setup(){
  pinMode(btn,INPUT_PULLUP);
  Serial.begin(9600);
}

void loop(){
  if (digitalRead(btn) == 0){
    if ((millis() - ultimapressao)>tempo){
      contador++;
    }
    ultimapressao = millis();
  }
  Serial.println(contador);
}
```

Variáveis:

Unsined long → variável que não aceita números negativos (maior variável)

Tempo → vai manter o tempo de pressão do botão.

Usando o millis → se o botão for pressionado, ele checa se a ultima vez que o botão foi pressionado é maior que o tempo, caso seja ele faz o que está dentro do IF do millis

Ultimapressao → sendo atualizada o momento que pressionamos o botão, com o tempo atual do microcontrolador.

Displays

Display LCD

Um display LCD (Liquid Crystal Display) capaz de exibir até 16 caracteres por linha em 2 linhas.
Usa uma interface paralela com múltiplos pinos para controle.

Características:

- Controlado por comandos e dados enviados pelo microcontrolador.
- Retroiluminação opcional.
- Ideal para exibir mensagens em projetos embarcados.

Desvantagens do Display Tradicional:

- Número elevado de pinos: Necessita de ao menos 6 pinos do microcontrolador (modo 4 bits).
- Complexidade: Mais conexões físicas e maior dificuldade em projetos com espaço limitado.

Display LCD – I2C

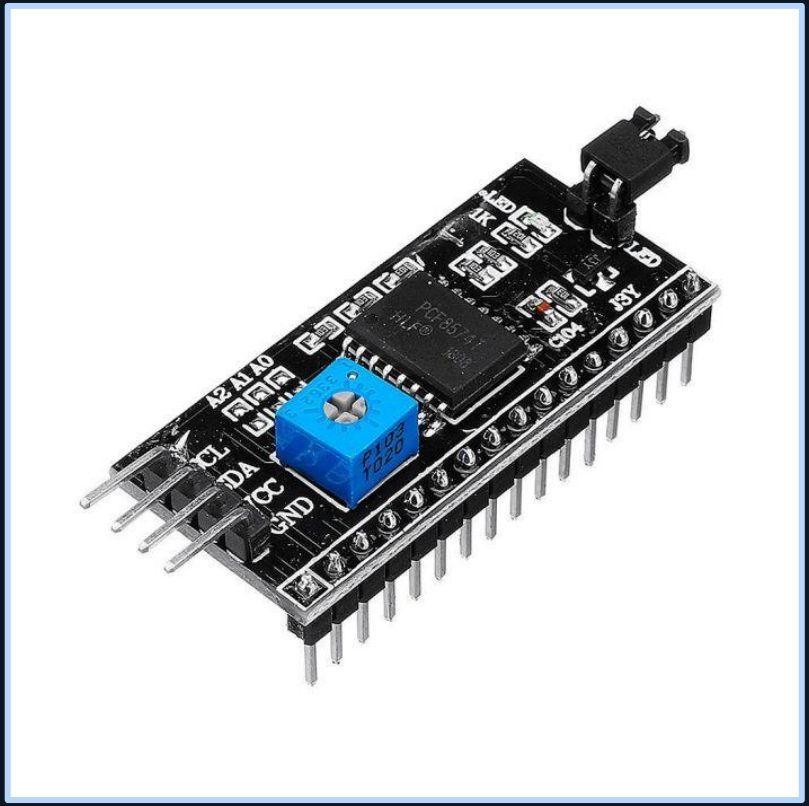
I2C (Inter-Integrated Circuit) é um protocolo de comunicação serial síncrono.
Ideal para conectar múltiplos dispositivos com o uso de apenas 2 fios.

Características:

- SDA (Dados): Transferência de informações.
- SCL (Clock): Sincronização do sinal.
- Cada dispositivo tem um endereço único.

Vantagens:

- Reduz drasticamente a quantidade de conexões necessárias.
- Suporte a múltiplos dispositivos no mesmo barramento.



Display LCD – I2C

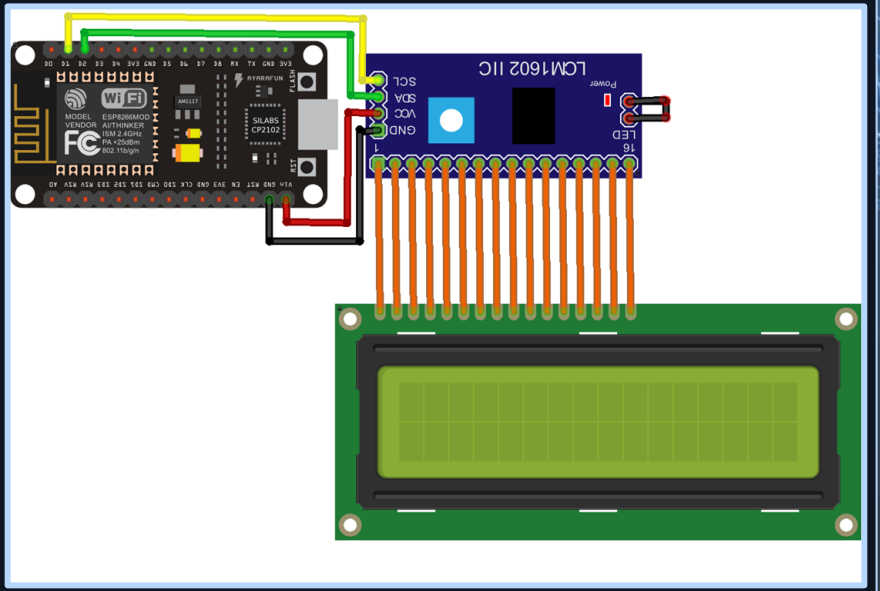
Um display LCD acoplado a um módulo I2C, que converte a interface paralela em uma interface serial.

Vantagens:

- Reduz a conexão a apenas 4 fios: VCC, GND, SDA, SCL.
- Fácil integração com microcontroladores como o ESP8266.

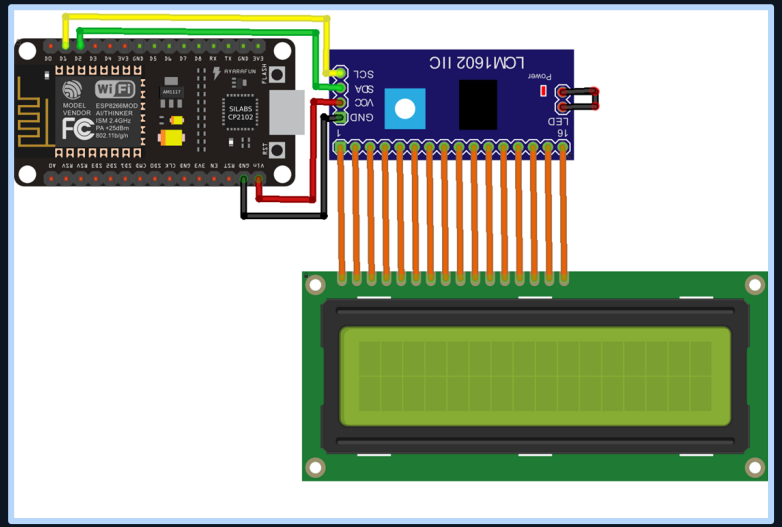
LCD I2C:

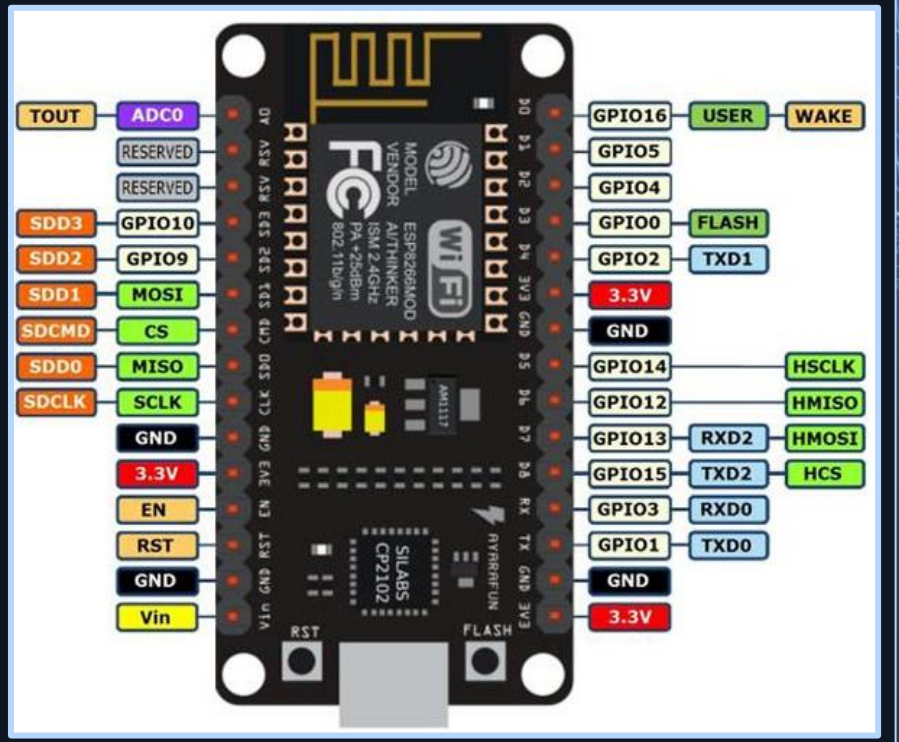
- VCC: Alimentação (3.3V ou 5V).
- GND: Terra.
- SDA: Dados (conectado ao GPIO4 no ESP8266).
- SCL: Clock (conectado ao GPIO5 no ESP8266).



1º Programa

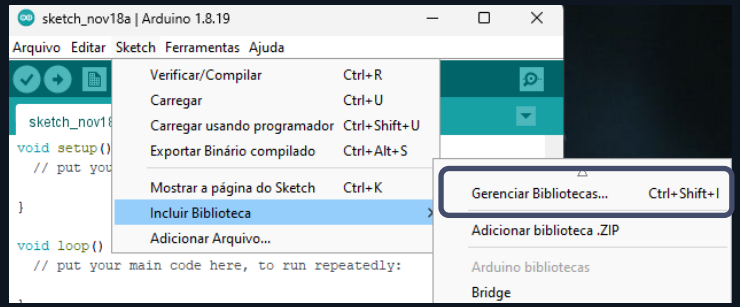
Vamos criar um programa para exibir uma mensagem conhecida no LCD. A mensagem deve ser: Hello World

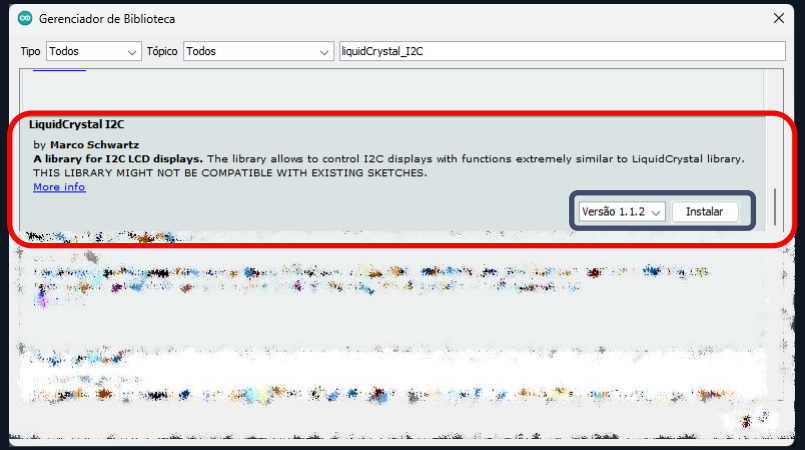




1º Programa

Para trabalhar com o LCD I2C vamos incluir uma biblioteca chamada LiquidCrystal_I2C





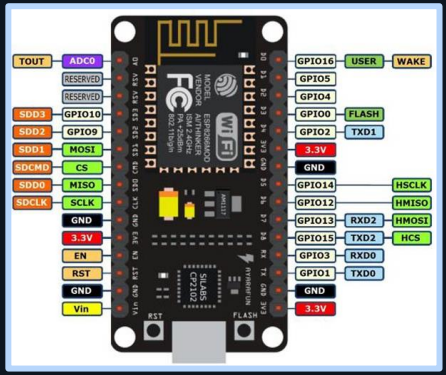
1º Programa

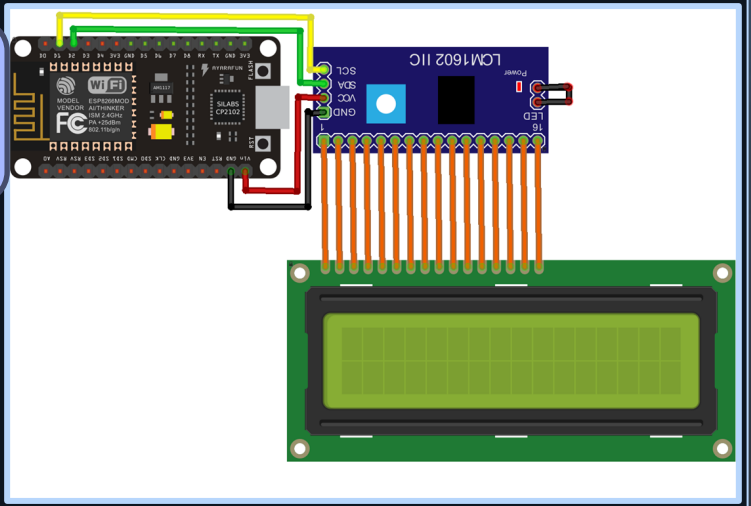
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Hello World!!");
}

void loop() {

}
```

Bibliotecas padrões para funcionamento do LCD

Define o Modelo do LCD

Inicia o LCD
Liga o LCD
Define a posição do cursor
Escreve o texto.





Pode ser que o LCD esteja muito claro, isso acontece pela tensão de entrada nele.

05

Exercícios de fixação.

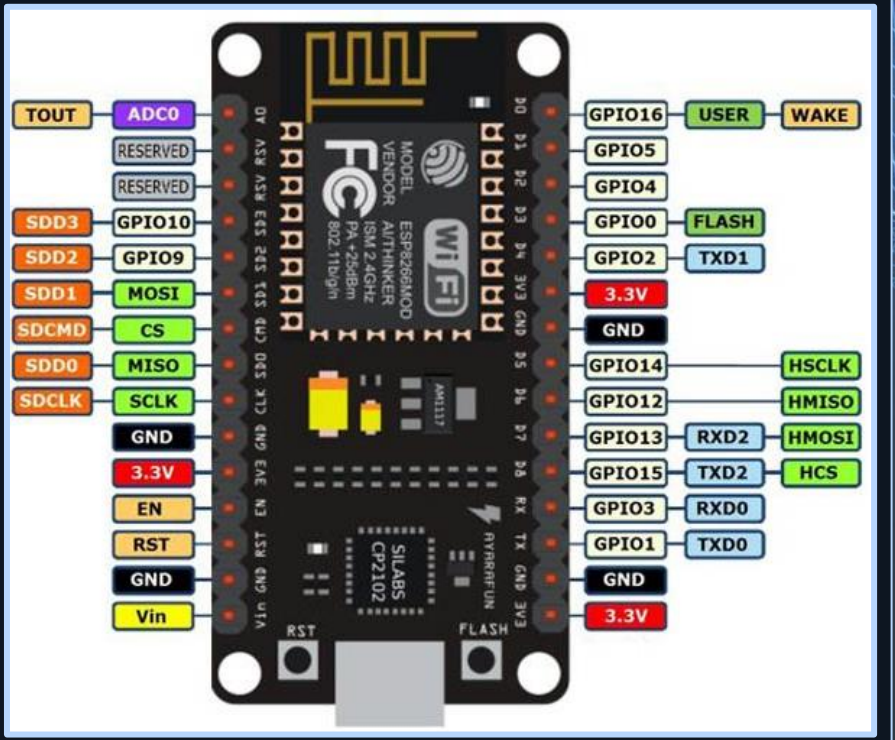
Exercício 01

Piscar 2 LEDs com Tempos Diferentes (Sem delay())

Objetivo:
Faça dois LEDs piscarem com tempos diferentes (ex.: LED1 a cada 1 segundo e LED2 a cada 500ms) sem usar delay().

Dica:

Use a função millis() para gerenciar o tempo.

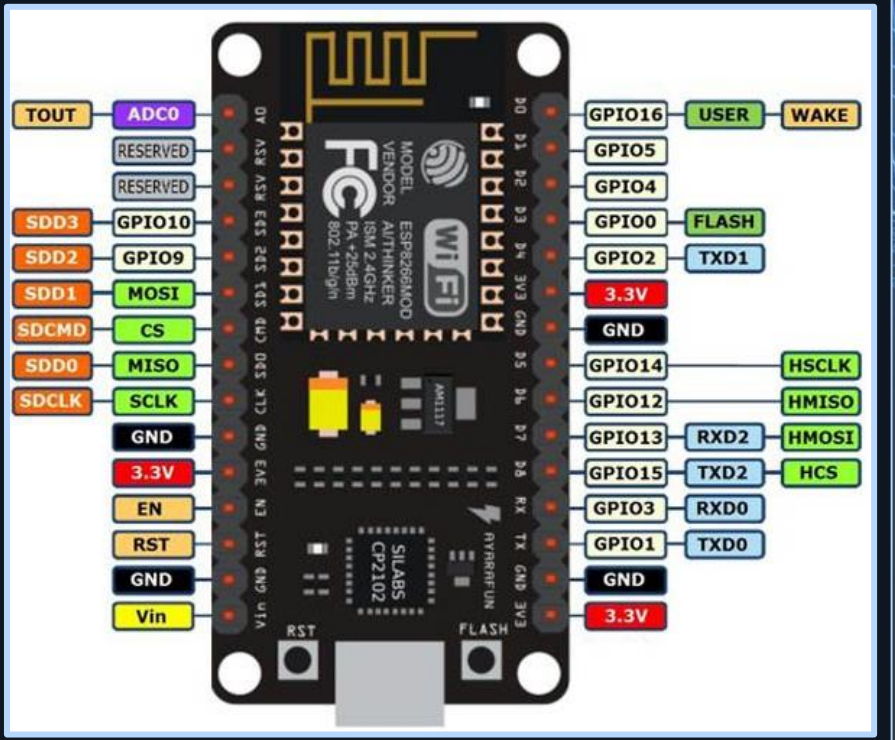


Exercício 02

Botão Controlando um LED

Objetivo:
Ao pressionar o botão 1x o LED deve ligar, pressionando novamente ele deve desligar.

Dica:
Use digitalRead() para ler o estado do botão e digitalWrite() para controlar o LED.
Use o debounce no botão.

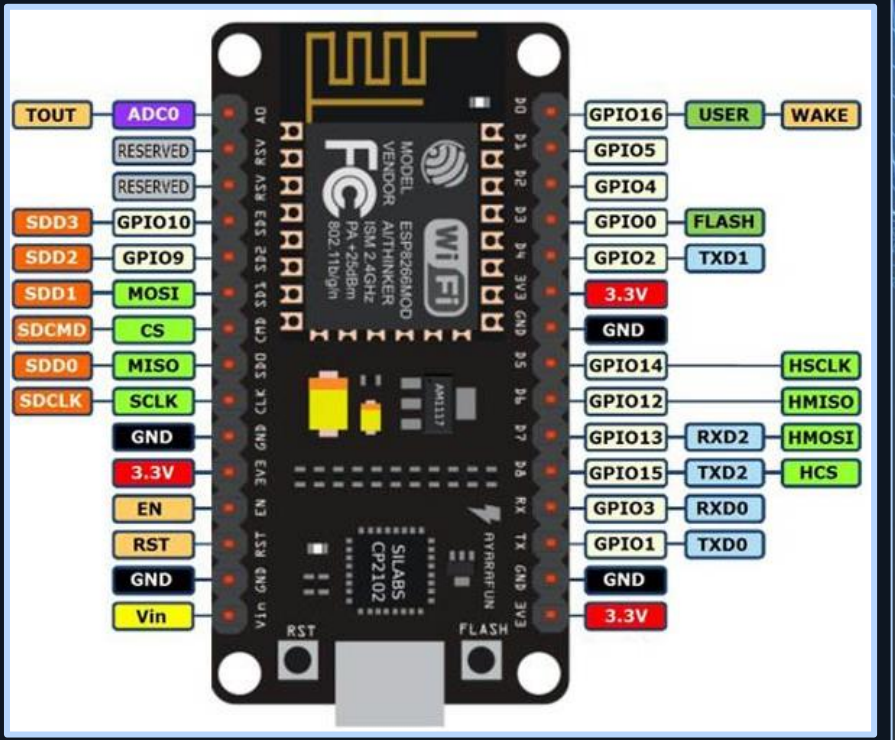


Exercício 03

Ajuste de Brilho com Potenciômetro

Objetivo:
Conecte um potenciômetro a uma entrada analógica e ajuste o brilho de um LED com base na leitura.

Dica:
Combine analogRead() e analogWrite().



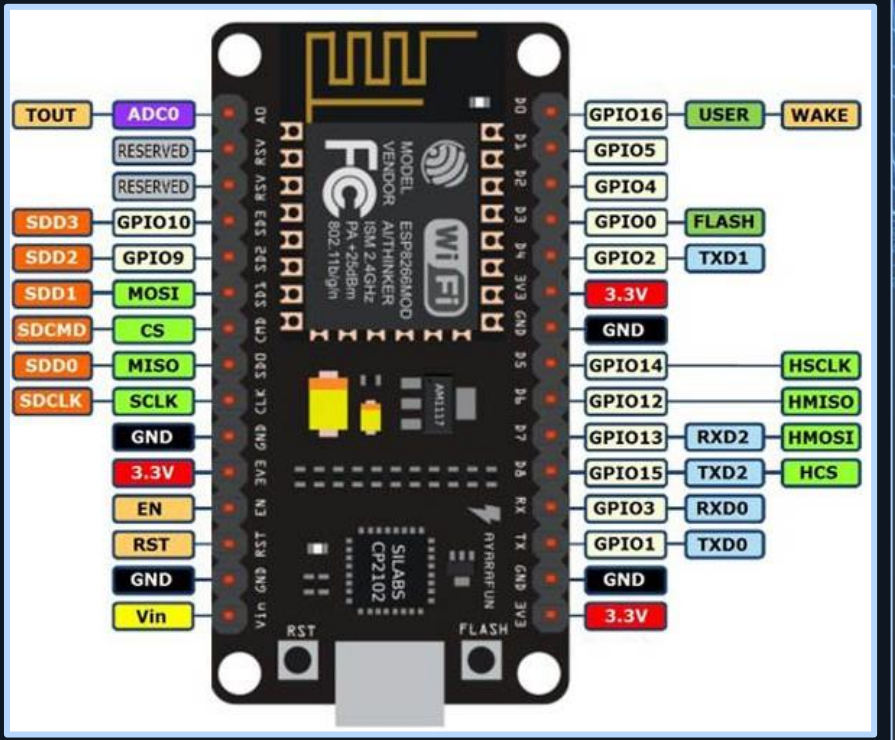
Exercício 04

Semáforo

Objetivo:
Controle 5 LEDs simulando um semáforo.

Carro:
Verde por 10 segundos, amarelo por 2 segundos e vermelho por 10 segundos.

Pedestre:
Verde por 8 segundos e vermelho piscando por 2 segundos antes de ficar fixo.



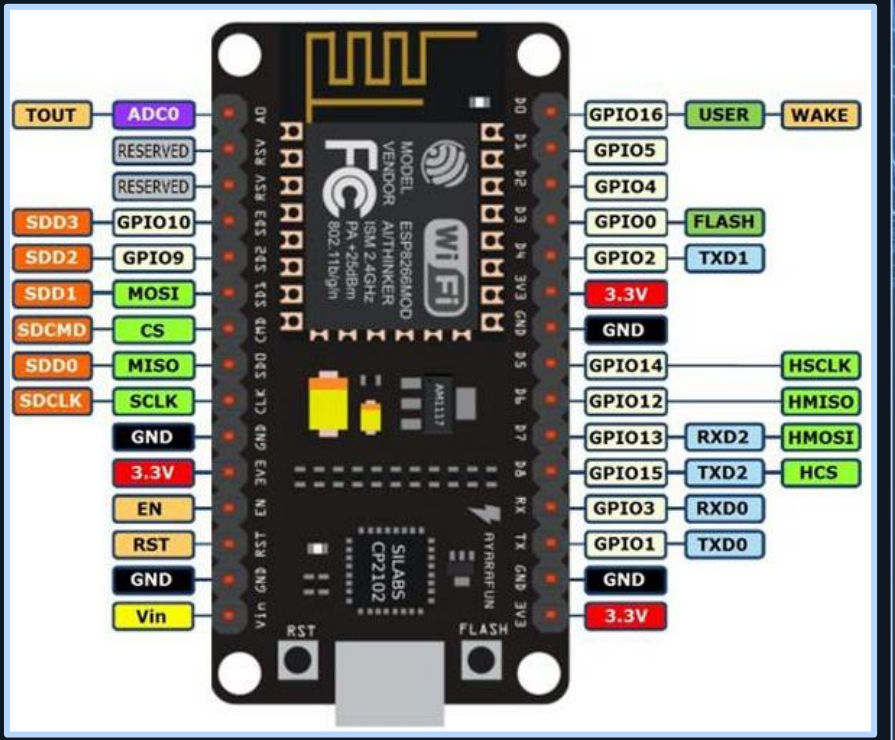
Dica:
Use millis() para gerenciar os tempos.

Exercício 05

Controle de LEDs com Dois Botões

Objetivo:
Um botão aumenta o brilho de um LED.O outro diminui o brilho.

Dica:
Use variáveis para armazenar o nível de brilho e analogWrite() para aplicar.

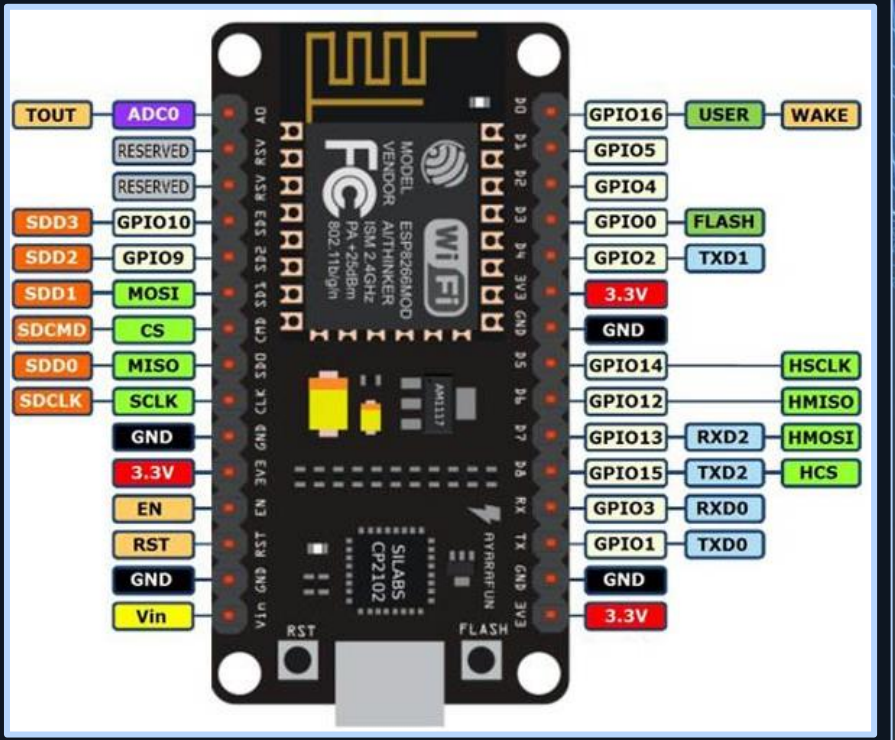


Exercício 06

Controle de LED com Display LCD

Objetivo:
Exiba "LED Ligado" ou "LED Desligado" em um display LCD com base no estado de um botão.

Dica:
Use a biblioteca LiquidCrystal_I2C para controlar o LCD.

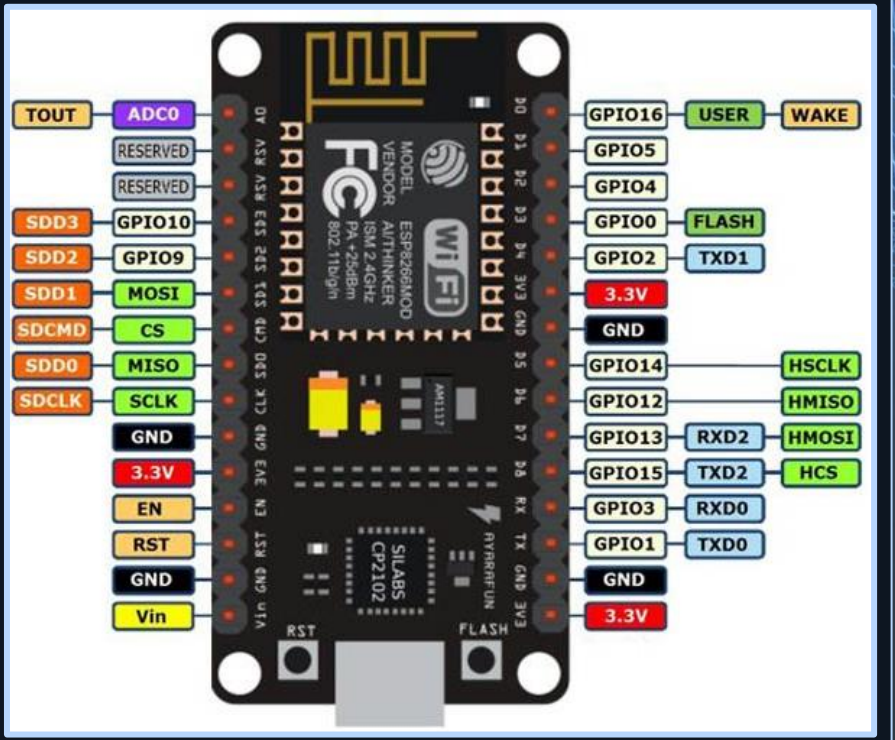


Exercício 07

Sequência de LEDs com Tempo Configurável

Objetivo:
Faça uma sequência de LEDs (um acende e apaga antes do próximo) e permita ajustar a velocidade com um potenciômetro.

Dica:
Use analogRead() para determinar o intervalo entre os LEDs.

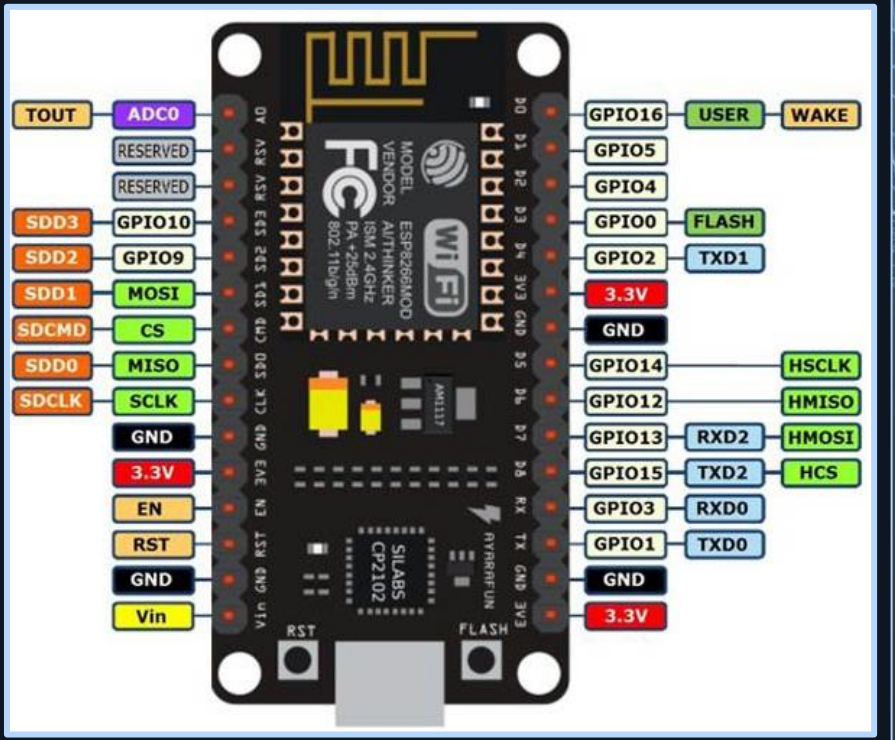


Exercício 08

Sensor de Proximidade Acionando LEDs

Objetivo:
Use um sensor de proximidade (ex.: ultrassônico) para acionar LEDs:
- Distância maior que 50 cm: LED verde aceso.
- Distância entre 20 e 50 cm: LED amarelo.
- Distância menor que 20 cm: LED vermelho.

Dica:
Converta a leitura do sensor em valores lógicos para acionar os LEDs.

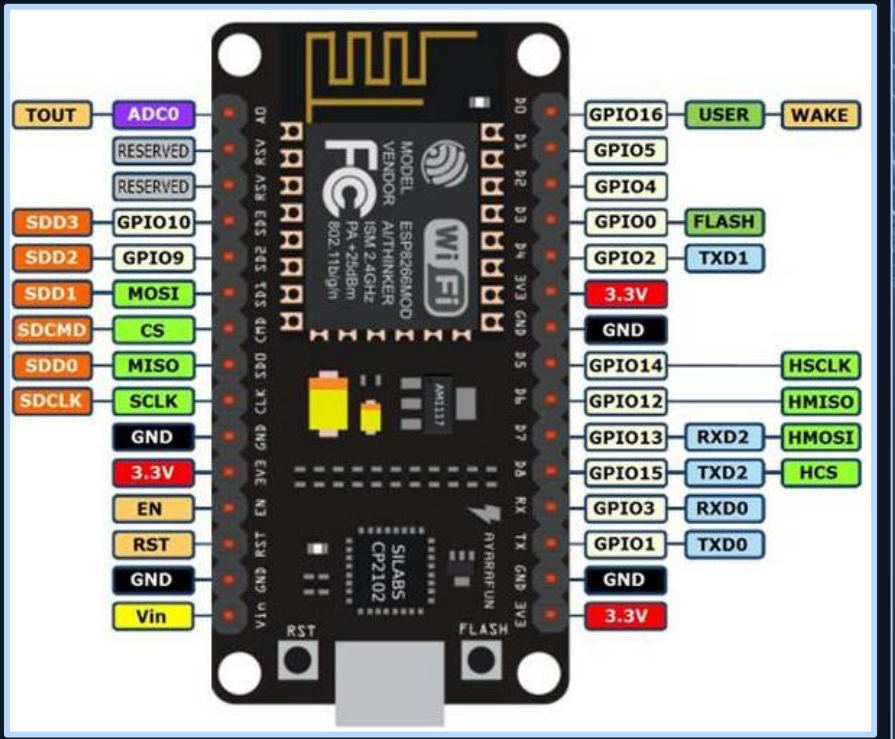


Exercício 09

Controle de Relé com Botão e Sensor

Objetivo:
Controle um relé com base em:
- Um botão liga/desliga manual.
- Um sensor de proximidade que ativa o relé automaticamente quando a distância for menor que 15 cm.

Dica:
Combine lógica de botões e sensores com condições if.

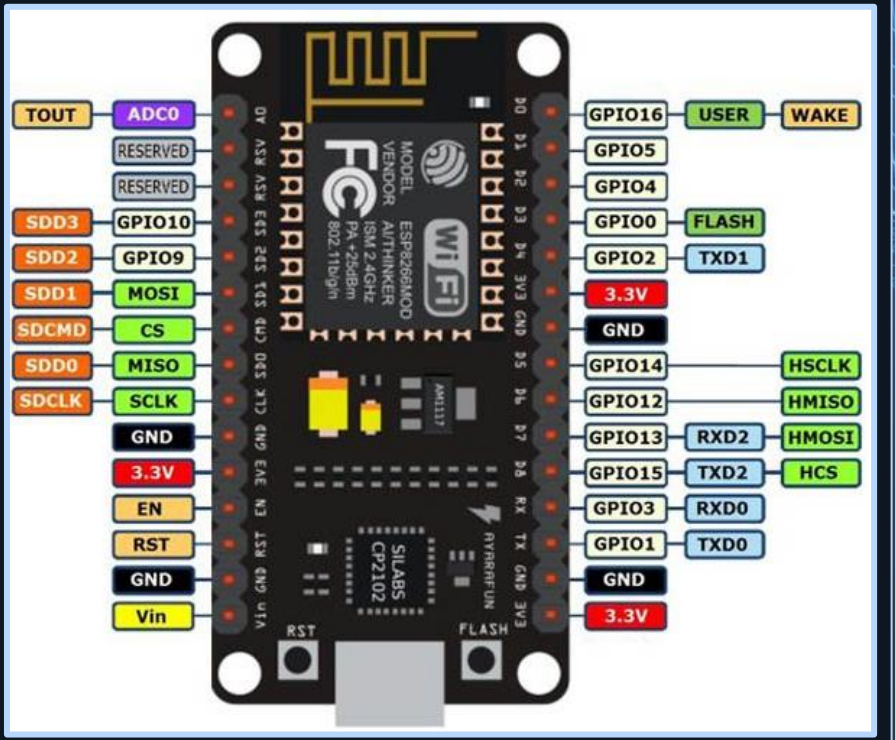


Exercício 10

Sistema de Alarme Simples

Objetivo:
Conecte um botão para armar/desarmar um sistema de alarme:
Quando armado, um sensor de proximidade ativa um buzzer e um LED vermelho.
Desarme o alarme pressionando o botão novamente.

Dica:
Use uma variável para rastrear o estado do sistema.



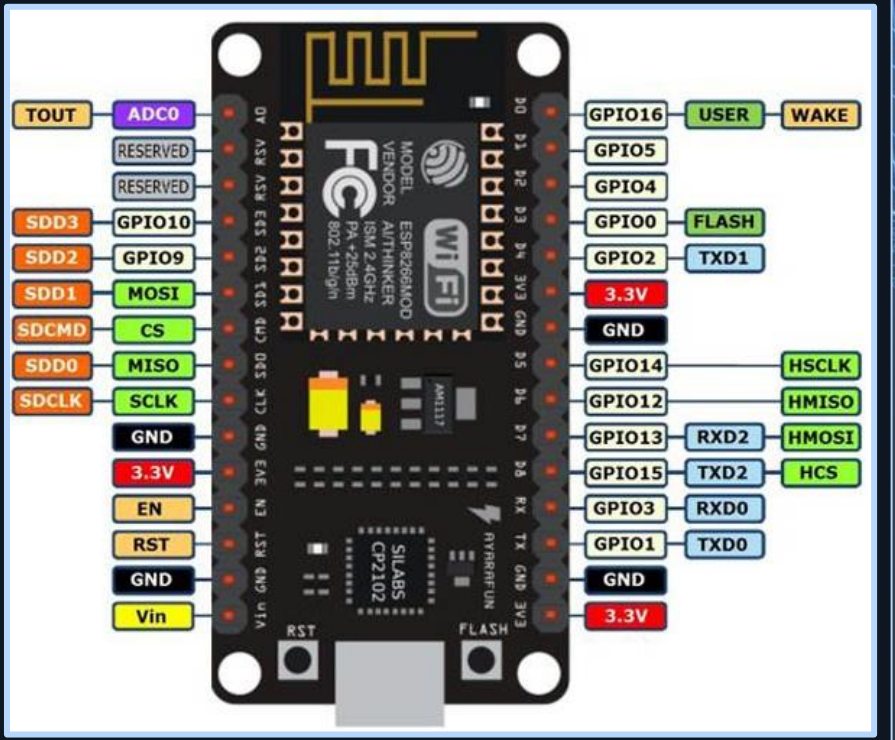
Exercício 11 - Desafio

Controle de LEDs via Comunicação Serial

Objetivo:
Controle 3 LEDs usando comandos enviados pelo Monitor Serial:
LED1 ON, LED1 OFF para controlar o LED1.
Similar para LED2 e LED3.

Dica:
Use Serial.readString() para interpretar os comandos.

Obs: Não explicado – Serve como desafio

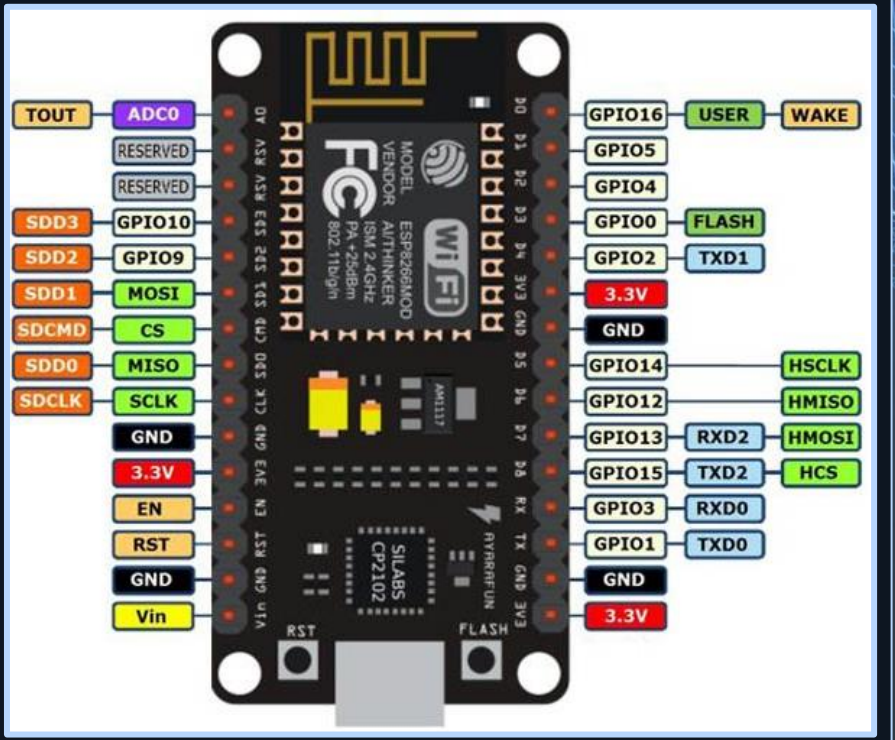


Exercício 12 - Desafio

Semáforo Inteligente com Sensor de Proximidade

Objetivo:
Use um sensor de proximidade para controlar um semáforo:
Se a distância for menor que 10 cm, alterne para o sinal vermelho.
Caso contrário, siga o ciclo normal (verde → amarelo → vermelho).

Dica:
Use millis() para gerenciar os tempos do semáforo.



Obrigado!